

Self-Supervised Learning (SSL) and Meta-Learning for Computer Vision

GenAI and Modern Deep Learning

Transfer Learning

Tasks can have limited data

Train one big model on tons of “pre-training” data

Pre-training data can be unrelated to the desired task

Fine-tune on downstream datasets with limited samples

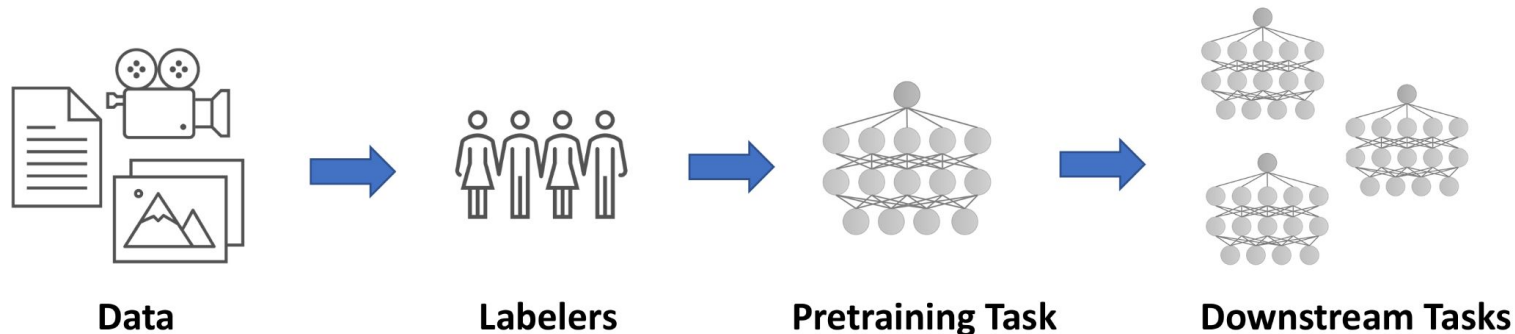
One “foundation model” for many downstream tasks

Pre-training data can be data that's easy to acquire or aggregated from multiple datasets

Transfer Learning

Typical transfer learning pipeline:

1. Collect tons of “pretraining” data
2. Label it - **often most expensive for very large webscale datasets**
3. Pre-train (e.g. natural image classification)
4. Rip “head” off the feature extractor, e.g. linear classification head
5. Initialize new “head”, e.g. linear classification layer for new downstream task
6. Fine-tune on downstream task data (e.g. medical image classification)



Why does transfer learning work?

A pretrained model can be a good initialization for downstream tasks

Easier to solve downstream task on top of a good representation

(recall that we re-use the feature extractor from the pretrained model)

How to learn good representations?

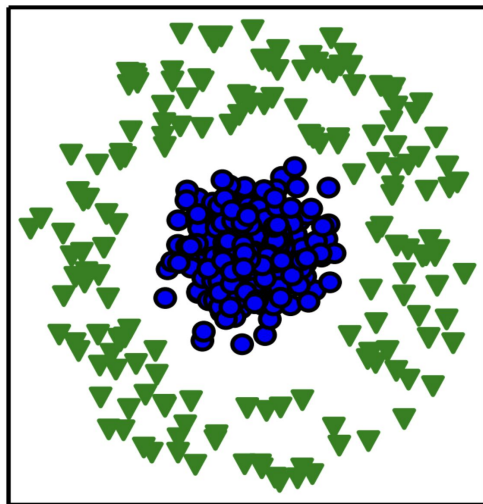
Good pretrained representation must be transferable

Representations Matter

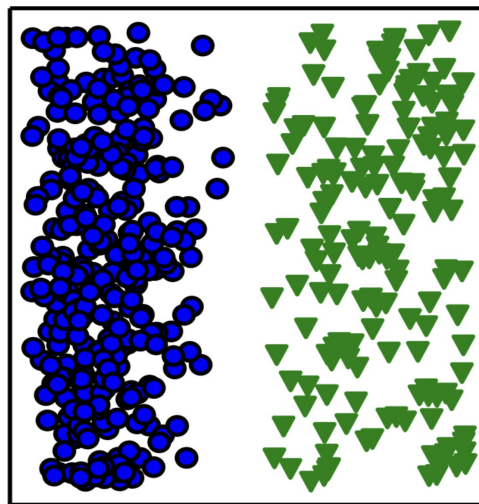
Left = raw data, right = output of pretrained feature extractor

Which would you rather train a linear classifier atop?

Cartesian coordinates



Polar coordinates



How to use pretrained models?

Linear probing: fix feature extractor, train linear or bigger head downstream

Full fine-tuning: place head on top of feature extractor, fine-tune everything including feature extractor

Linear heads

Complicated heads (e.g. object detectors)

Pretrained model as part of bigger pipeline

How to use pretrained models?

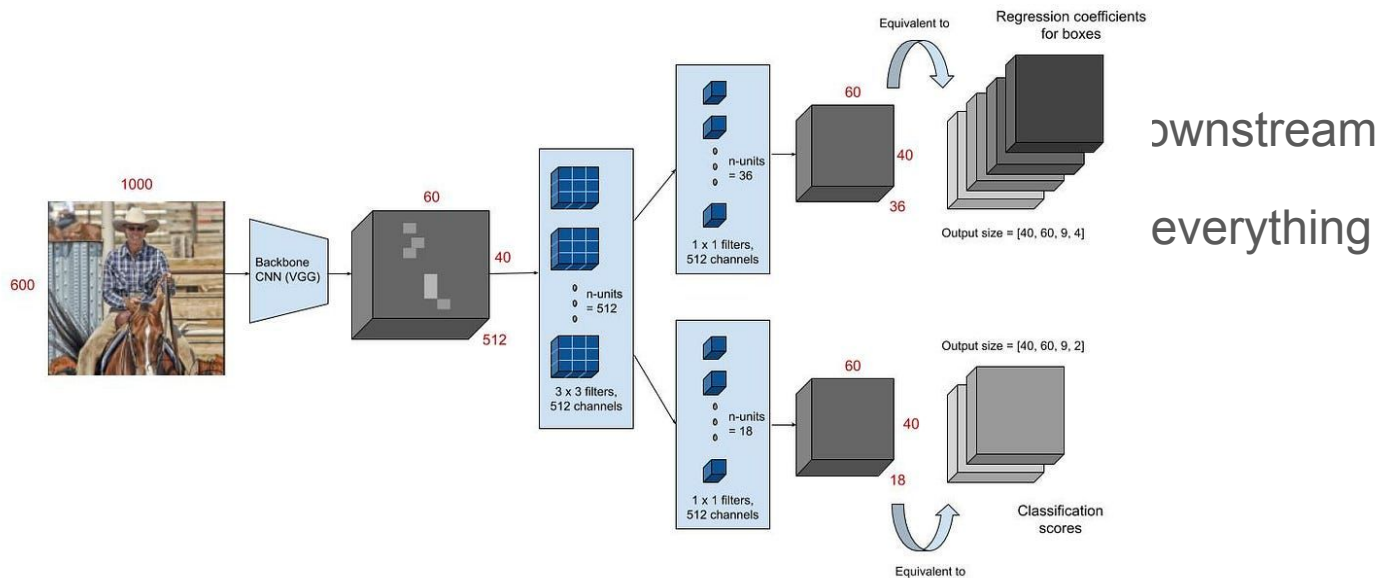
Linear pro

Full fine-t
including

Linear he

Complica

Pretrained model as part of bigger pipeline



Want to do transfer learning

How best to pretrain a model?

Want to learn transferable representations

Today, we'll learn about strategies for learning transferable representations.

Let's start with supervised learning

Representation Learning with Labels

Start with labeled pre-training dataset - let's consider image classification

How to pretrain?

Most obvious solution - train with standard image classification loss

Can we do better?

How we pretrain might depend on what we want to do with the pretrained model

Few-Shot Learning

Goal: perform downstream tasks with few samples

Can leverage pretrained model on downstream tasks

Example: want to identify rare species of insects, but we can pretrain on massive amounts of data for common species of insects

Few-Shot Learning - Classification Setup

Pretrain on some classes

Fine-tune on different classes

Fine-tune on k samples from each of n different classes

k -shot n -way classification

Fine-tuning classes typically disjoint from pretraining classes

Want what we learned on the pretraining classes to be useful for downstream task

Few-shot classification is a popular eval for SSL and meta-learning

Example Dataset: Mini-ImageNet

Take ImageNet dataset

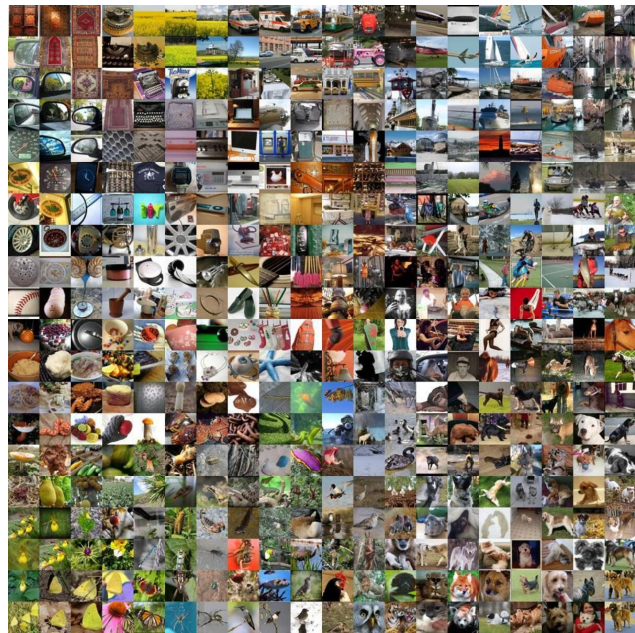
Only 100 random classes from ImageNet

600 images per class (84 x 84)

64 training classes (for pretraining)

16 validation classes (hyperparameter tuning)

20 test classes



Meta-Learning

Can we pretrain a model specifically to be good at few-shot classification?

Meta-learning: learning to learn

We'll focus on few-shot classification, but meta-learning is used in reinforcement learning, optimization, model selection, and more.

In few-shot learning:

Train model explicitly so that it's good when fine-tuned on few samples

Meta-Learning: A Blueprint

Meta-learning has two general steps

Inner loop: simulate fine-tuning (or another inference procedure) on randomly sampled few-shot classification tasks

Outer loop: compute loss on held out data, nudge initial params so that fine-tuning results in lower loss on held out samples from same classes.

Outer loop is just gradient descent.

Requires computing gradient through inner loop.

To invent a new meta-learning algorithm, propose a new inner loop.

Meta-Learning: Some Terminology

In each step of meta-learning, we sample a task (or batch of task)

Task - n classes out of total pretraining classes

Support - sample 1 or more images from task to fine-tune on in the inner loop

Query - sample 1 or more images from task to evaluate on in the outer loop

Meta-Learning: A Blueprint

Algorithm 1 The meta-learning framework

Require: Base model, F_θ , fine-tuning algorithm, A , learning rate, γ , and distribution over tasks, $p(\mathcal{T})$.

Initialize θ , the weights of F ;

while not done **do**

Sample batch of tasks, $\{\mathcal{T}_i\}_{i=1}^n$, where $\mathcal{T}_i \sim p(\mathcal{T})$ and $\mathcal{T}_i = (\mathcal{T}_i^s, \mathcal{T}_i^q)$.

for $i = 1, \dots, n$ **do**

Fine-tune model on $\mathcal{T}_i$ (inner loop). New network parameters are written $\theta_i = A(\theta, \mathcal{T}_i^s)$.

Compute gradient $g_i = \nabla_\theta \mathcal{L}(F_{\theta_i}, \mathcal{T}_i^q)$

end for

Update base model parameters (outer loop):

$\theta \leftarrow \theta - \frac{\gamma}{n} \sum_i g_i$

end while

MAML (Finn 2017)

Inner loop: gradient descent (fine-tuning) on support data

Gradient descent is a function, input initial params and outputs fine-tuned params

$$g(\theta) = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\text{inner}}(f_{\theta})$$

Change initial params -> change fine-tuned params

How will changing initial params change loss computed on finetuned params?

$$\nabla_{\theta} \mathcal{L}_{\text{outer}}(f_{g(\theta)})$$

Standard deep learning libraries like PyTorch support second-order gradients!

MAML (Finn 2017)

Famous meta-learning algorithm

Break training loop into 4 steps:

1. Sample n -class task from pretraining data
2. Sample k points per class (called **support**) and testing data (called **query**)
3. Gradient descent on the $k*n$ support samples
4. Compute cross-entropy loss on the query samples
5. Nudge parameters so that fine-tuning on support will minimize loss on query

That's one iteration

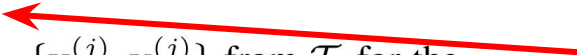
Repeat many times!

MAML (Finn 2017)

Algorithm 2 MAML for Few-Shot Supervised Learning

Require: $p(\mathcal{T})$: distribution over tasks

Require: α, β : step size hyperparameters

- 1: randomly initialize θ
 - 2: **while** not done **do**
 - 3: Sample batch of tasks $\mathcal{T}_i \sim p(\mathcal{T})$
 - 4: **for all** $\mathcal{T}_i$ **do**
 - 5: Sample K datapoints $\mathcal{D} = \{\mathbf{x}^{(j)}, \mathbf{y}^{(j)}\}$ from $\mathcal{T}_i$  Support data
 - 6: Evaluate $\nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$ using $\mathcal{D}$ and $\mathcal{L}_{\mathcal{T}_i}$ in Equation (2) or (3)
 - 7: Compute adapted parameters with gradient descent:
 $\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$  Fine-tuning
One or more steps
 - 8: Sample datapoints $\mathcal{D}'_i = \{\mathbf{x}^{(j)}, \mathbf{y}^{(j)}\}$ from $\mathcal{T}_i$ for the meta-update
 - 9: **end for**
 - 10: Update $\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i})$ using each $\mathcal{D}'_i$ and $\mathcal{L}_{\mathcal{T}_i}$ in Equation 2 or 3  Loss on query data
 - 11: **end while**
-

MAML (Finn 2017)

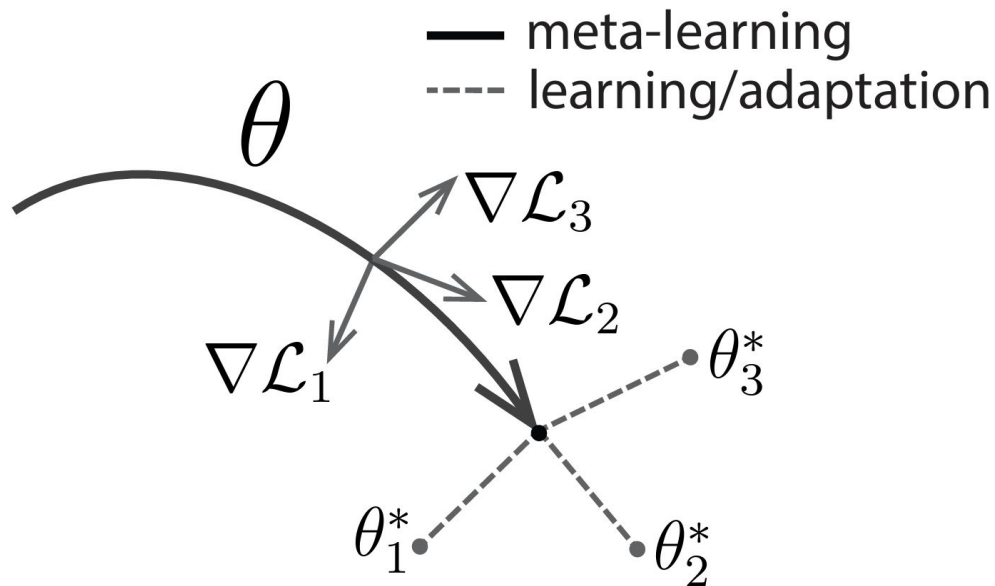


Figure 1. Diagram of our model-agnostic meta-learning algorithm (MAML), which optimizes for a representation θ that can quickly adapt to new tasks.

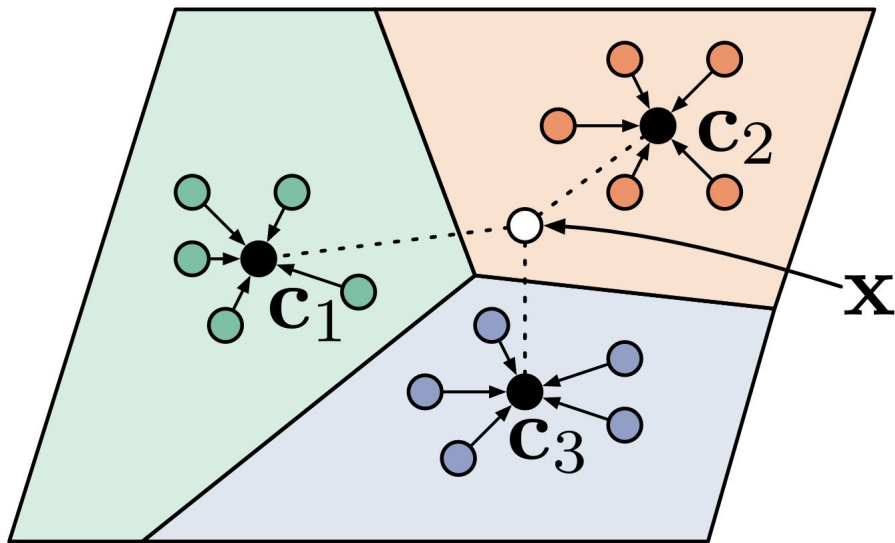
MAML (Finn 2017)

MiniImagenet (Ravi & Larochelle, 2017)	5-way Accuracy	
	1-shot	5-shot
fine-tuning baseline	$28.86 \pm 0.54\%$	$49.79 \pm 0.79\%$
nearest neighbor baseline	$41.08 \pm 0.70\%$	$51.04 \pm 0.65\%$
matching nets (Vinyals et al., 2016)	$43.56 \pm 0.84\%$	$55.31 \pm 0.73\%$
meta-learner LSTM (Ravi & Larochelle, 2017)	$43.44 \pm 0.77\%$	$60.60 \pm 0.71\%$
MAML, first order approx. (ours)	$48.07 \pm 1.75\%$	$63.15 \pm 0.91\%$
MAML (ours)	$48.70 \pm 1.84\%$	$63.11 \pm 0.92\%$

ProtoNets (Snell 2017)

New inference procedure:

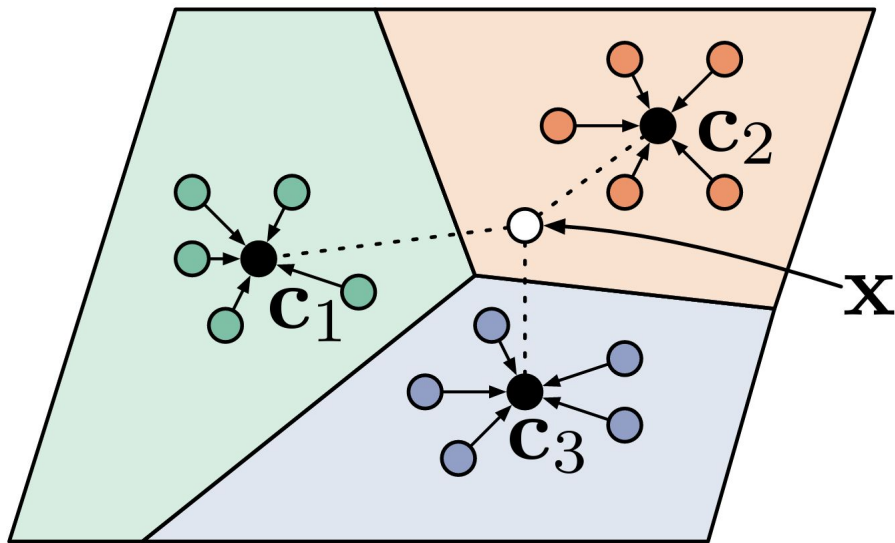
- (1) Compute average of features for each class (class prototype)
- (2) Classify new sample via nearest neighbors to class prototypes



ProtoNets (Snell 2017)

How to do the meta-learning?

New inner loop: treat Euclidean distance to class prototypes as logits, use softmax to compute class probabilities and cross-entropy classification loss.



ProtoNets (Snell 2017)

Algorithm 2 MAML for Few-Shot Supervised Learning

Require: $p(\mathcal{T})$: distribution over tasks

Require: α, β : step size hyperparameters

```
1: randomly initialize  $\theta$ 
2: while not done do
3:   Sample batch of tasks  $\mathcal{T}_i \sim p(\mathcal{T})$ 
4:   for all  $\mathcal{T}_i$  do
5:     Sample  $K$  datapoints  $\mathcal{D} = \{\mathbf{x}^{(j)}, \mathbf{y}^{(j)}\}$  from  $\mathcal{T}_i$ 
6:     Evaluate  $\nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$  using  $\mathcal{D}$  and  $\mathcal{L}_{\mathcal{T}_i}$  in Equation (2)
       or (3)
7:     Compute adapted parameters with gradient descent:
        $\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$ 
8:     Sample datapoints  $\mathcal{D}'_i = \{\mathbf{x}^{(j)}, \mathbf{y}^{(j)}\}$  from  $\mathcal{T}_i$  for the
       meta-update
9:   end for
10:  Update  $\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i})$  using each  $\mathcal{D}'_i$ 
    and  $\mathcal{L}_{\mathcal{T}_i}$  in Equation 2 or 3
11: end while
```

Swap out the fine-tuning
for class prototypes

Distance from query
features to class
prototypes

ProtoNets (Snell 2017)

Table 2: Few-shot classification accuracies on *miniImageNet*. All accuracy results are averaged over 600 test episodes and are reported with 95% confidence intervals. *Results reported by [22].

Model	Dist.	Fine Tune	5-way Acc.	
			1-shot	5-shot
BASELINE NEAREST NEIGHBORS*	Cosine	N	$28.86 \pm 0.54\%$	$49.79 \pm 0.79\%$
MATCHING NETWORKS [29]*	Cosine	N	$43.40 \pm 0.78\%$	$51.09 \pm 0.71\%$
MATCHING NETWORKS FCE [29]*	Cosine	N	$43.56 \pm 0.84\%$	$55.31 \pm 0.73\%$
META-LEARNER LSTM [22]*	-	N	$43.44 \pm 0.77\%$	$60.60 \pm 0.71\%$
PROTOTYPICAL NETWORKS (OURS)	Euclid.	N	$49.42 \pm 0.78\%$	$68.20 \pm 0.66\%$

MiniImagenet (Ravi & Larochelle, 2017)	5-way Accuracy	
	1-shot	5-shot
fine-tuning baseline	$28.86 \pm 0.54\%$	$49.79 \pm 0.79\%$
nearest neighbor baseline	$41.08 \pm 0.70\%$	$51.04 \pm 0.65\%$
matching nets (Vinyals et al., 2016)	$43.56 \pm 0.84\%$	$55.31 \pm 0.73\%$
meta-learner LSTM (Ravi & Larochelle, 2017)	$43.44 \pm 0.77\%$	$60.60 \pm 0.71\%$
MAML, first order approx. (ours)	$48.07 \pm 1.75\%$	$63.15 \pm 0.91\%$
MAML (ours)	$48.70 \pm 1.84\%$	$63.11 \pm 0.92\%$

Limitations of Meta-Learning

- Meta-learning can overfit on big models, SOTA achieved with tiny convnets
- Doesn't scale, two-step procedure can be memory and time consuming
- Train model for very specific downstream inference procedure

Supervised Learning

Supervised learning: dataset contains (input, label) pairs

Supervised learning loss functions: push model's output towards label

- Cross-entropy
- Mean-squared error



ImageNet (Deng et al. 2009)
1k class version, ~1M images
21k class version, ~15M images
Different breeds of dogs,
household objects, etc.

But... supervised learning comes with a cost!

Collecting labels is expensive!

ImageNet took 3 years to label

49k Amazon MechanicalTurkers

Certain types of data (e.g. radiology) requires experts to label



ImageNet (Deng et al. 2009)
1k class version, ~1M images
21k class version, ~15M images
Different breeds of dogs,
household objects, etc.

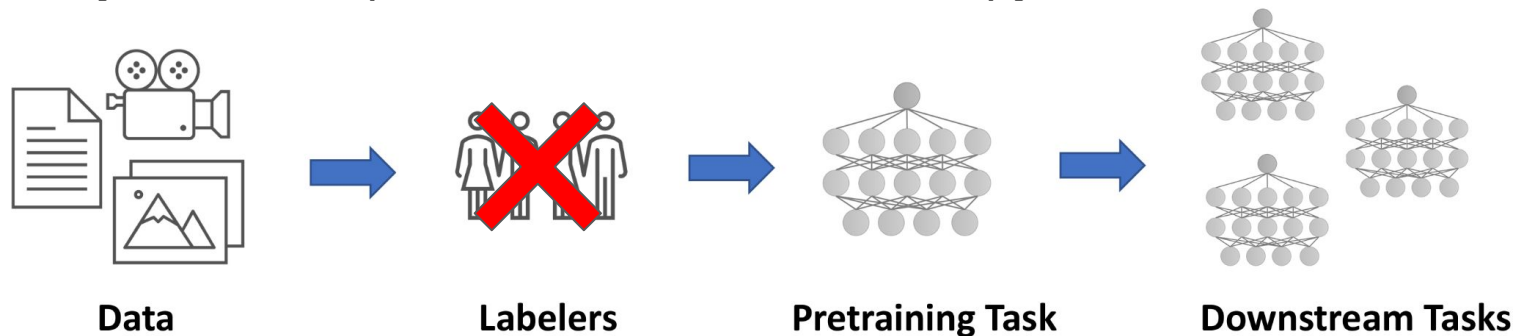
Self-Supervised Learning

What if we had no labels?

Could we still learn anything?

Self-supervised learning:

- Invent your own learning problem
- Synthesize labels automatically
- Ideally, still use supervised loss like cross-entropy or MSE



Why do self-supervised learning?

We have tons and tons of data in the world

The vast majority of it is unlabeled

How to leverage unlabeled data?

Example:

Put dash cam on car, take tons and tons of pictures - **cheap**

Draw bounding boxes around objects - **expensive**

How to use just the pictures without bounding boxes to learn feature extractor?

Why do self-supervised learning? - Yann LeCun

- ▶ **“Pure” Reinforcement Learning (cherry)**

- ▶ The machine predicts a scalar reward given once in a while.

- ▶ **A few bits for some samples**

- ▶ **Supervised Learning (icing)**

- ▶ The machine predicts a category or a few numbers for each input

- ▶ Predicting human-supplied data

- ▶ **10→10,000 bits per sample**

- ▶ **Self-Supervised Learning (cake génoise)**

- ▶ The machine predicts any part of its input for any observed part.

- ▶ Predicts future frames in videos

- ▶ **Millions of bits per sample**



Yann LeCun's cake

Why do self-supervised learning?

Summary:

- Can get way more data (don't need to label)
- Squeeze more juice out of each sample (Yann's layer cake)

Self-Supervised Learning in NLP

LLMs train on massive corpora of text

The text doesn't have labels

Next-token prediction is SSL

Vision SSL usually looks different, but can do similar stuff there too

Approaches to SSL

- Reconstructing corrupted images
- Autoregressive
- Common-sense visual tasks
- Contrastive learning
- Feature prediction and self-distillation
- Text-image (CLIP)

Approaches to SSL

- Reconstructing corrupted images
- Autoregressive
- Common-sense visual tasks
- Contrastive learning
- Feature prediction and self-distillation
- Text-image (CLIP)

Reconstructing Corrupted Images

One archetype for vision SSL:

Apply corruption to images

Plug corrupted images into model

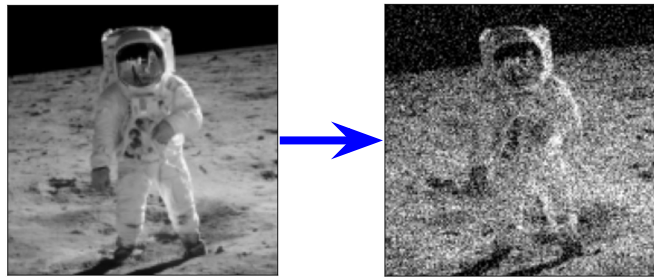
Teach model to undo the corruption

Denoising Autoencoders (Vincent 2008)

Simple corruption: Gaussian noise

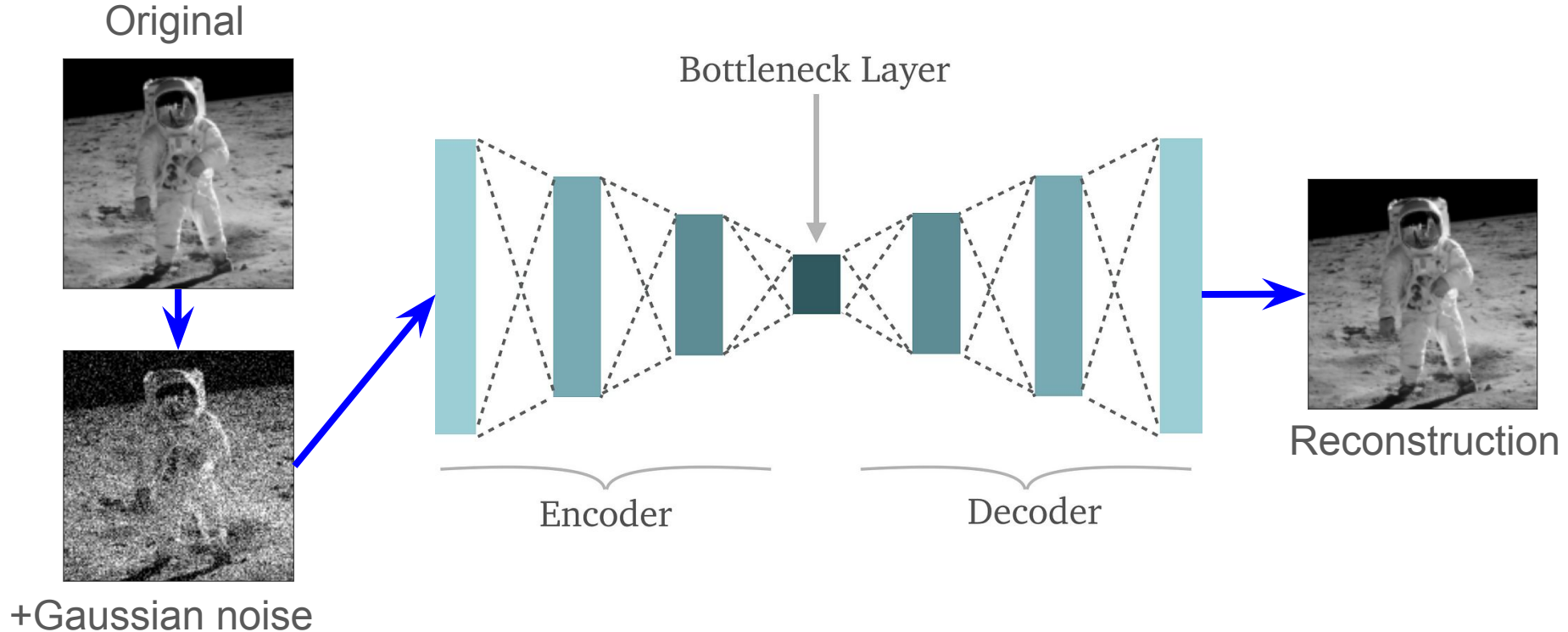
Add Gaussian noise to images

Train model to denoise the images



Loss-function: pixel-wise mean-squared error

Denoising Autoencoders (Vincent 2008)



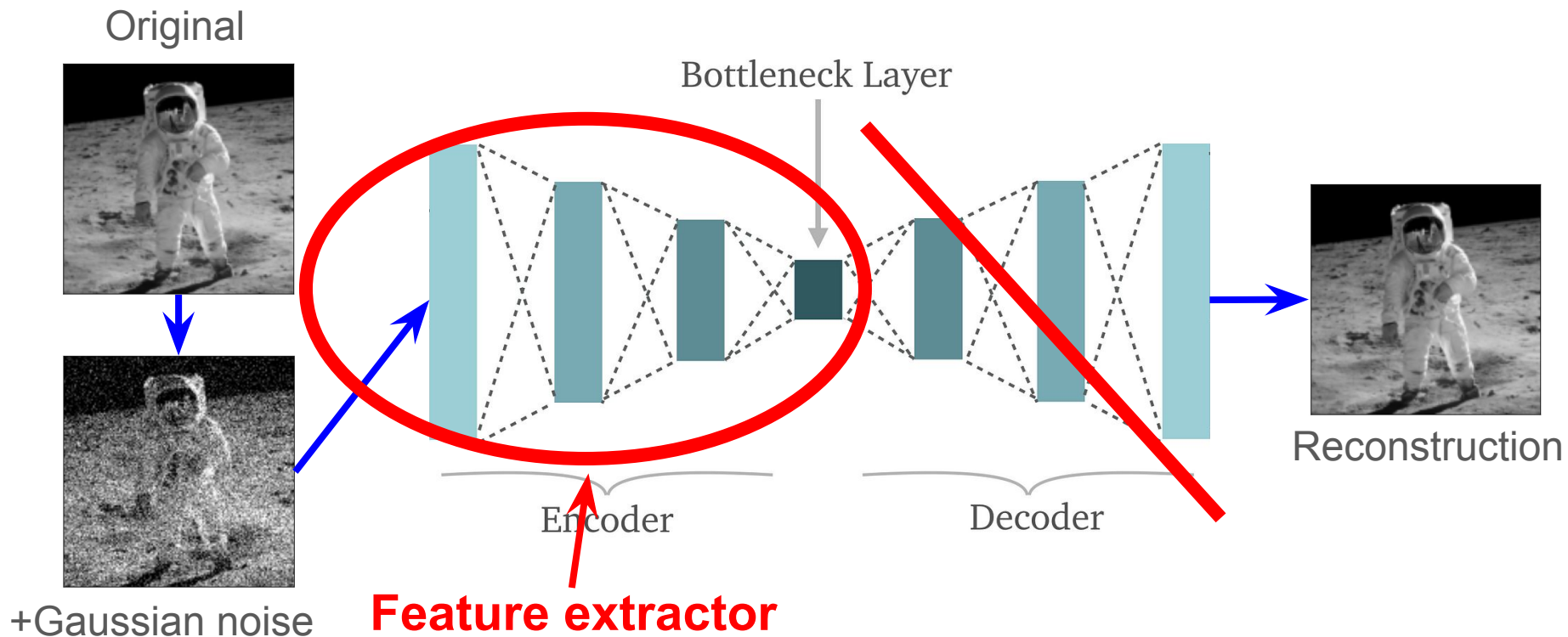
Denoising Autoencoders (Vincent 2008)

So now we have a model that can denoise images

How do we get a feature extractor out of it?

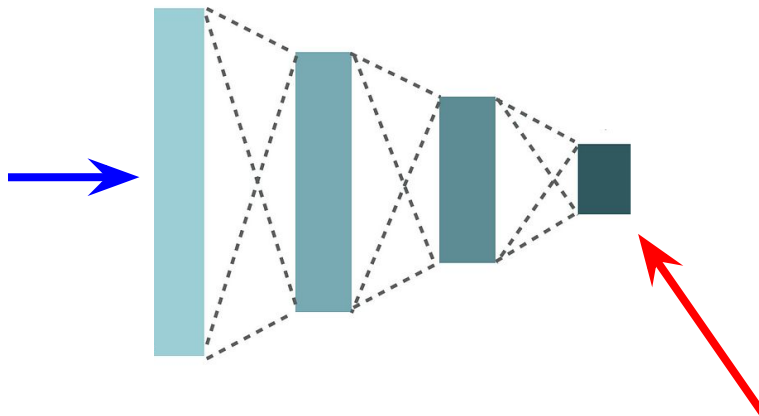
Rip off the decoder, keep encoder

Denoising Autoencoders (Vincent 2008)



Denoising Autoencoders (Vincent 2008)

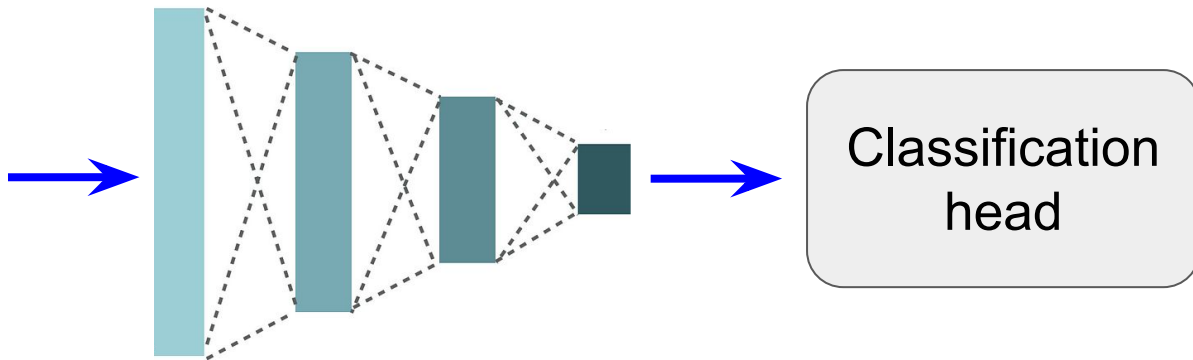
For new downstream tasks, use pretrained feature extractor (encoder) to extract features



Features

Denoising Autoencoders (Vincent 2008)

Put new head on top of pretrained feature extractor
Fine-tune



Denoising Autoencoders (Vincent 2008)

A critical limitation:

Denoising autoencoders are taught to extract features from noisy images

Natural images are not noisy

Model unequipped for natural images

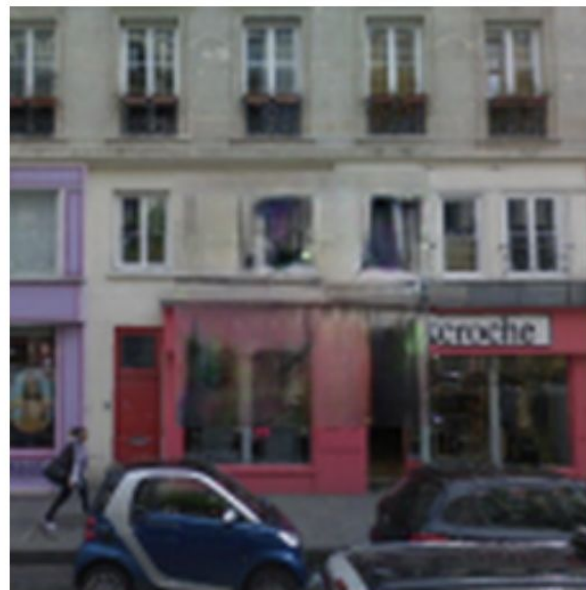
Reconstructing Corrupted Images

Some possible corruptions:

- Gaussian noise
- Salt and pepper noise (randomly set pixels to max or min value)
- Masking
- ...



Feature Learning by Inpainting (Pathak 2016)



Autoencoder, input masked image, output reconstruction

Feature Learning by Inpainting (Pathak 2016)

$$\mathcal{L}_{rec}(x) = \|\hat{M} \odot (x - F((1 - \hat{M}) \odot x))\|_2^2$$

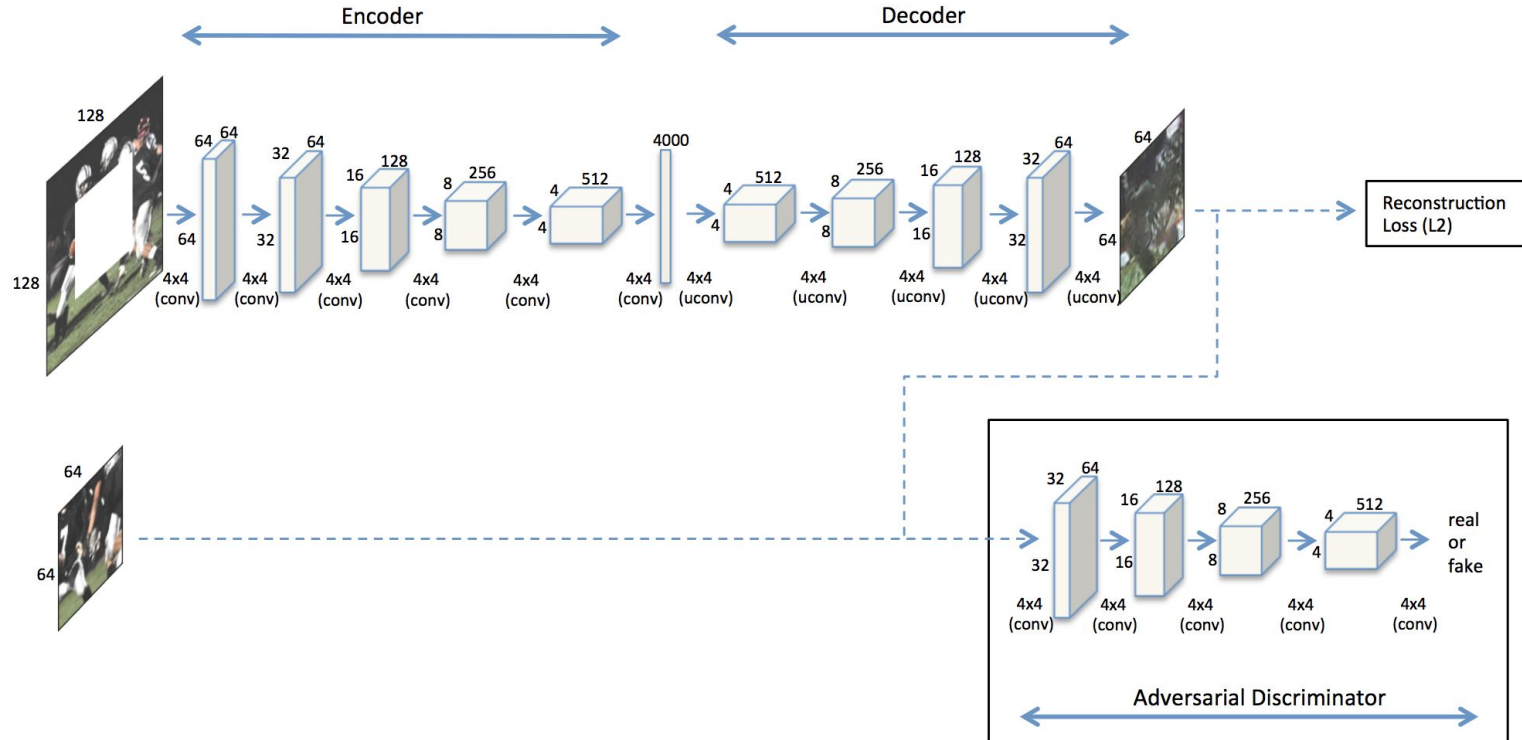
$$\mathcal{L}_{adv} = \max_D \mathbb{E}_{x \in \mathcal{X}} [\log(D(x)) + \log(1 - D(F((1 - \hat{M}) \odot x)))]$$

$$\mathcal{L} = \lambda_{rec} \mathcal{L}_{rec} + \lambda_{adv} \mathcal{L}_{adv}$$

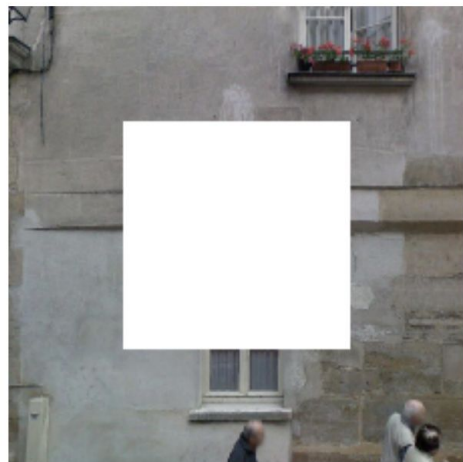
MSE leads to blurry outputs
Adversarial loss sharpens images

Train discriminator D to tell reconstruction vs. original
Train F to fool D

Feature Learning by Inpainting (Pathak 2016)



Feature Learning by Inpainting (Pathak 2016)



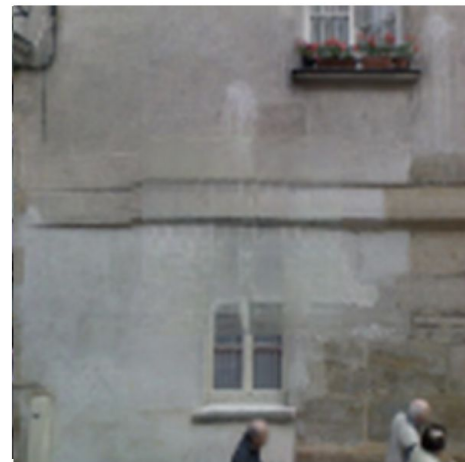
Input Image



L2 Loss



Adversarial Loss



Joint Loss

Feature Learning by Inpainting (Pathak 2016)

Pretraining Method	Supervision	Pretraining time	Classification	Detection	Segmentation
ImageNet [26]	1000 class labels	3 days	78.2%	56.8%	48.0%
Random Gaussian	initialization	< 1 minute	53.3%	43.4%	19.8%
Autoencoder	-	14 hours	53.8%	41.9%	25.2%
Agrawal <i>et al.</i> [1]	egomotion	10 hours	52.9%	41.8%	-
Doersch <i>et al.</i> [7]	context	4 weeks	55.3%	46.6%	-
Wang <i>et al.</i> [39]	motion	1 week	58.4%	44.0%	-
Ours	context	14 hours	56.5%	44.5%	29.7%

Table 2: Quantitative comparison for classification, detection and semantic segmentation. Classification and Fast-RCNN Detection results are on the PASCAL VOC 2007 test set. Semantic segmentation results are on the PASCAL VOC 2012 validation set from the FCN evaluation described in Section 5.2.3, using the additional training data from [18], and removing overlapping images from the validation set [28].

Efficient but much worse than supervised ImageNet pretraining

A lot of work done by decoder

Model is used to masked inputs

Masked Autoencoders (He 2021)

BERT but for images

Vision transformers encoder (overview on next slide)

Mask out patches of an image

Pass non-masked patches into transformer, extract features

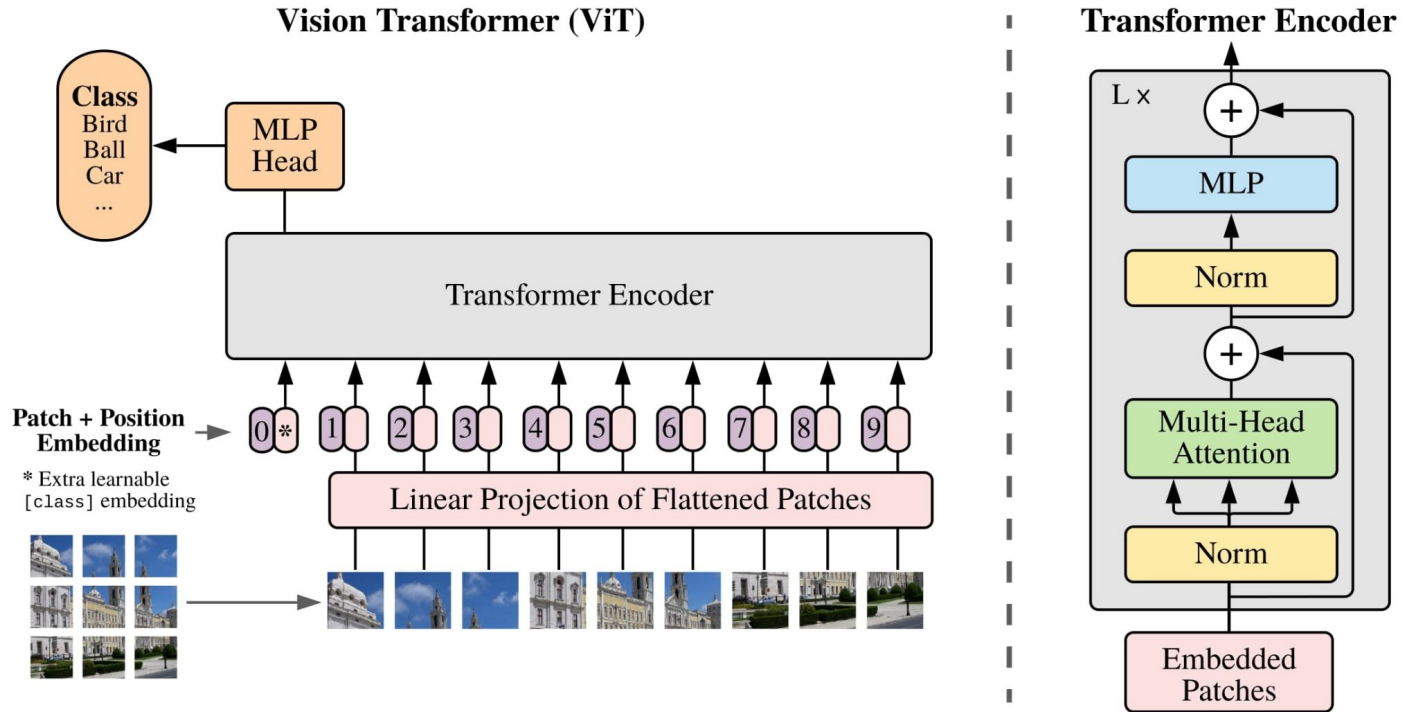
Decoder (small transformer) takes in features + mask tokens, reconstructs image

Regression, mean-squared error

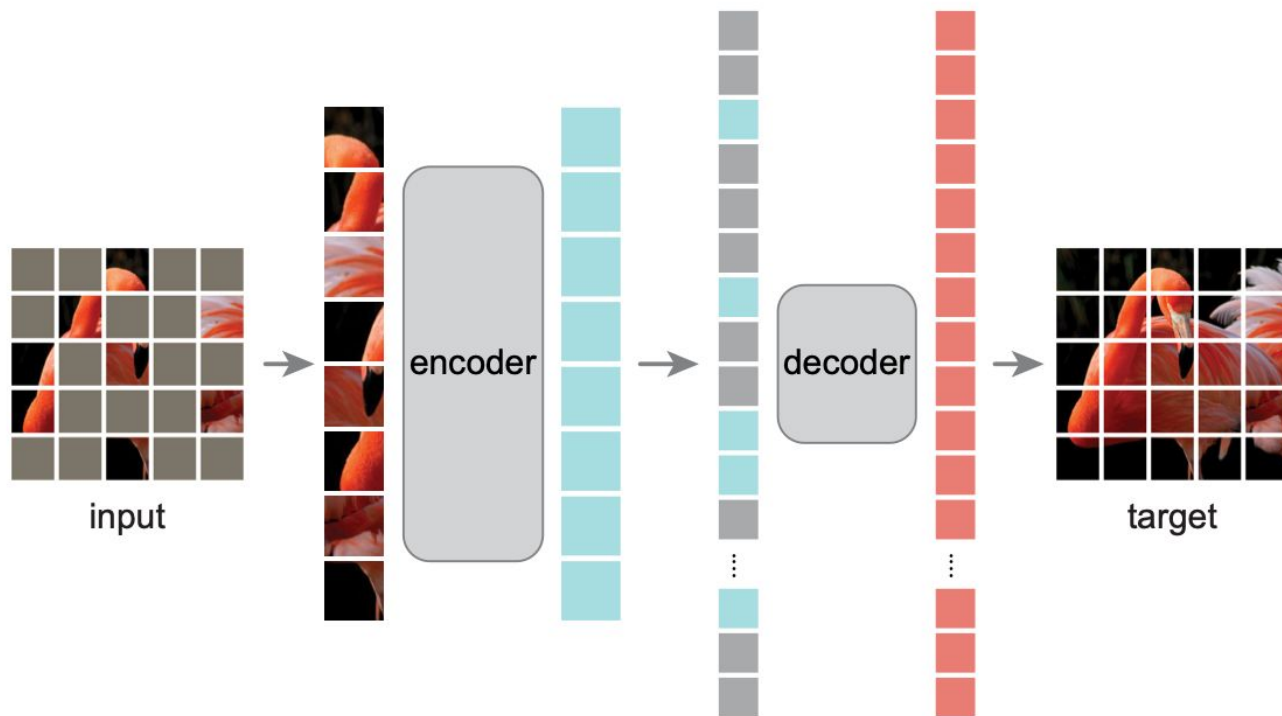
Rip off the decoder after pretraining, use encoder

MAE strategy is generalizable beyond images (e.g. audio, video, etc.)

Vision Transformers (Dosovitskiy 2020)



Masked Autoencoders (He 2021)



Masked Autoencoders (He 2021)

Images are redundant, tolerate very high masking ratios

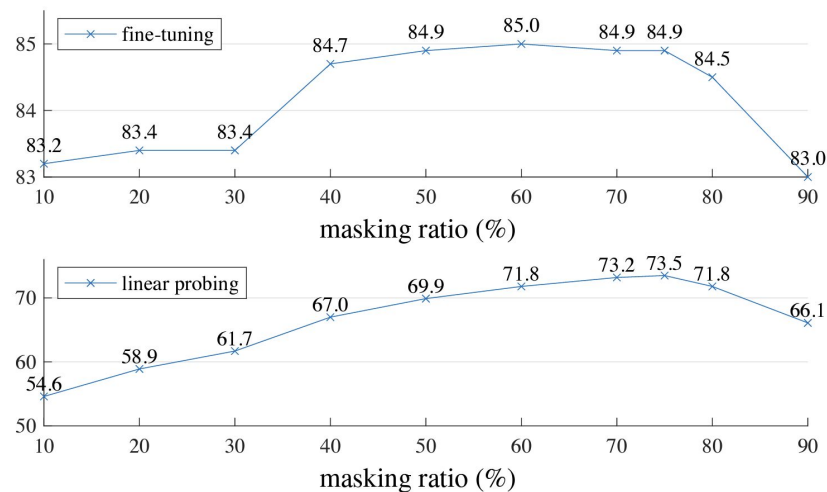


Figure 5. **Masking ratio.** A high masking ratio (75%) works well for both fine-tuning (top) and linear probing (bottom). The y-axes are ImageNet-1K validation accuracy (%) in all plots in this paper.

Masked Autoencoders (He 2021)

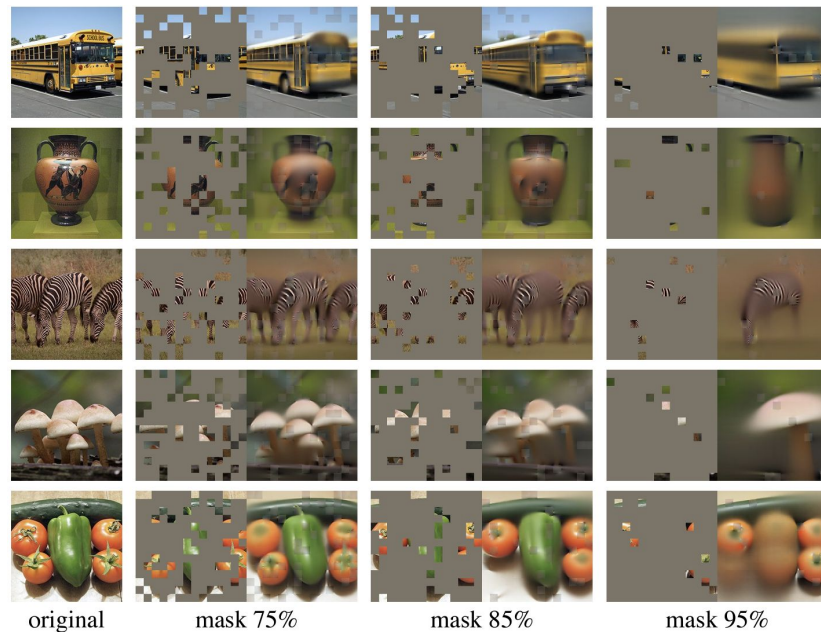


Figure 4. Reconstructions of ImageNet *validation* images using an MAE pre-trained with a masking ratio of 75% but applied on inputs with higher masking ratios. The predictions differ plausibly from the original images, showing that the method can generalize.

Masked Autoencoders (He 2021)

method	pre-train data	ViT-B	ViT-L	ViT-H	ViT-H ₄₄₈
scratch, our impl.	-	82.3	82.6	83.1	-
DINO [5]	IN1K	82.8	-	-	-
MoCo v3 [9]	IN1K	83.2	84.1	-	-
BEiT [2]	IN1K+DALLE	83.2	85.2	-	-
MAE	IN1K	<u>83.6</u>	<u>85.9</u>	<u>86.9</u>	87.8

Table 3. **Comparisons with previous results on ImageNet-1K.** The pre-training data is the ImageNet-1K training set (except the tokenizer in BEiT was pre-trained on 250M DALLE data [50]). All self-supervised methods are evaluated by end-to-end fine-tuning. The ViT models are B/16, L/16, H/14 [16]. The best for each column is underlined. All results are on an image size of 224, except for ViT-H with an extra result on 448. Here our MAE reconstructs normalized pixels and is pre-trained for 1600 epochs.

Approaches to SSL

- Reconstructing corrupted images
- Autoregressive
- Common-sense visual tasks
- Contrastive learning
- Feature prediction and self-distillation
- Text-image (CLIP)

Autoregressive Pretraining (El-Nouby 2024)

Autoregressive LLMs -> next-token prediction

Autoregressive vision models -> next-patch prediction

Causal masking is a corruption

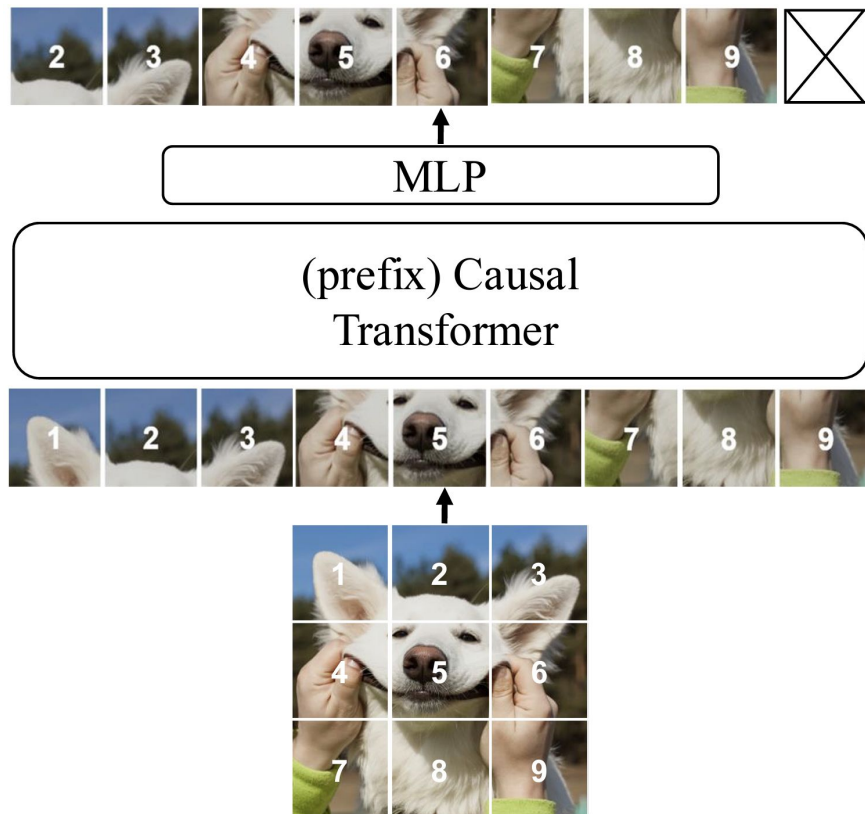
Find that prefix attention leads to more transferable models (more in a sec)

Generative modeling objective

Use to pretrain feature extractor

Use full self-attention downstream

Autoregressive Pretraining (El-Nouby 2024)



Autoregressive Pretraining (El-Nouby 2024)

Prefix causal attention

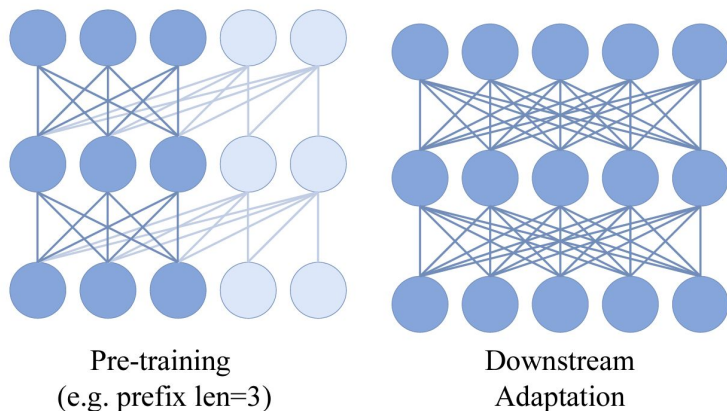


Figure 3. Prefix causal attention. During pre-training we uniformly sample a prefix length S . The attention for the first S patches are set to be bidirectional and loss is only computed for the remaining patches in the image. During adaptation to downstream tasks, this allows us to drop the attention causal mask, improving the downstream performance.

Autoregressive Pretraining (El-Nouby 2024)

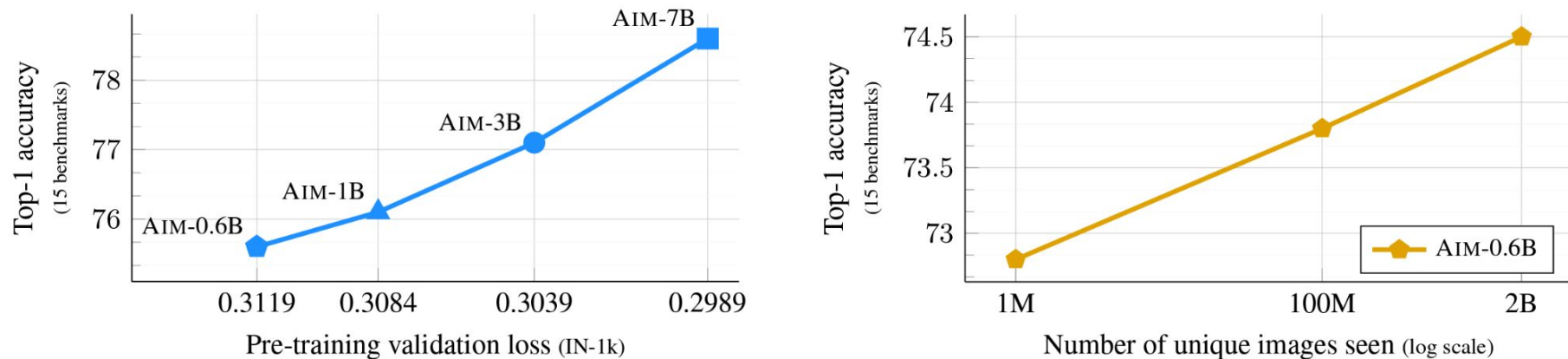


Figure 1. AIM scaling behavior (Left) As we scale the capacity of AIM, we observe improved performance for the pre-training objective which directly correlates with stronger downstream performance. (Right) AIM exhibits stronger downstream performance when trained using larger sets of uncured web data [32, 33]. The downstream performance is the average attentive probe top-1 accuracy over a diverse set of **15 image recognition benchmarks**. All models are trained for the same number of updates.

Approaches to SSL

- Reconstructing corrupted images
- Autoregressive
- **Common-sense visual tasks**
- Contrastive learning
- Feature prediction and self-distillation
- Text-image (CLIP)

Common-Sense Visual Tasks

Autoencoder pretraining involves decoders that get thrown out.
Instead of outputting image, let's output attribute of image

Often called “pretext tasks”

Old methods, not that popular anymore

Have been used more recently as add-ons to other methods

Jigsaw (Noroozi 2016)

Break an image into patches

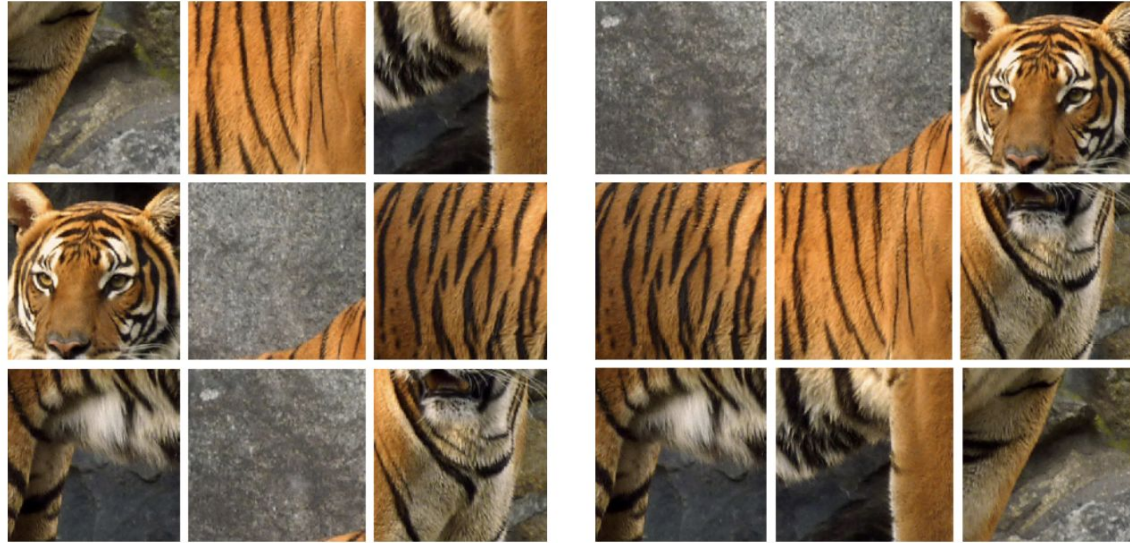
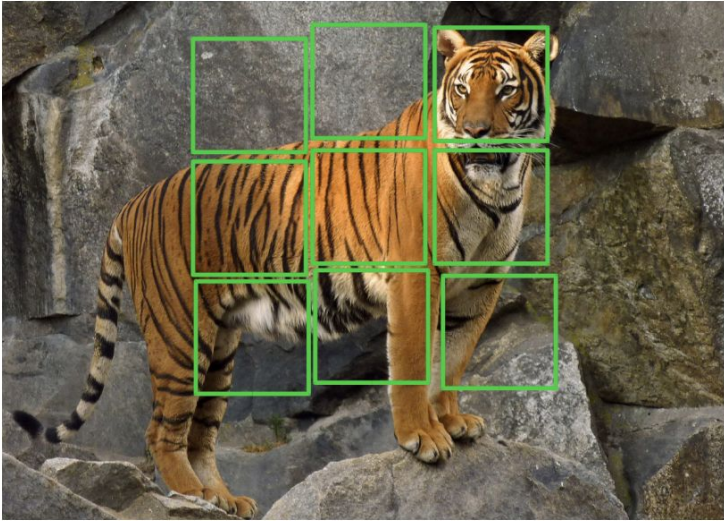
Shuffle patches

Pass each patch through a shared feature extractor

Use features to classify which patch belongs where with small MLP

Throw out MLP, use shared feature extractor downstream

Jigsaw (Noroozi 2016)

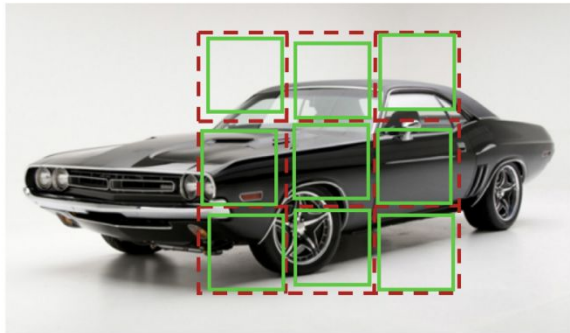


Jiggle patches and leave space between them

Can no longer just learn to match edges

Jigsaw (Noroozi 2016)

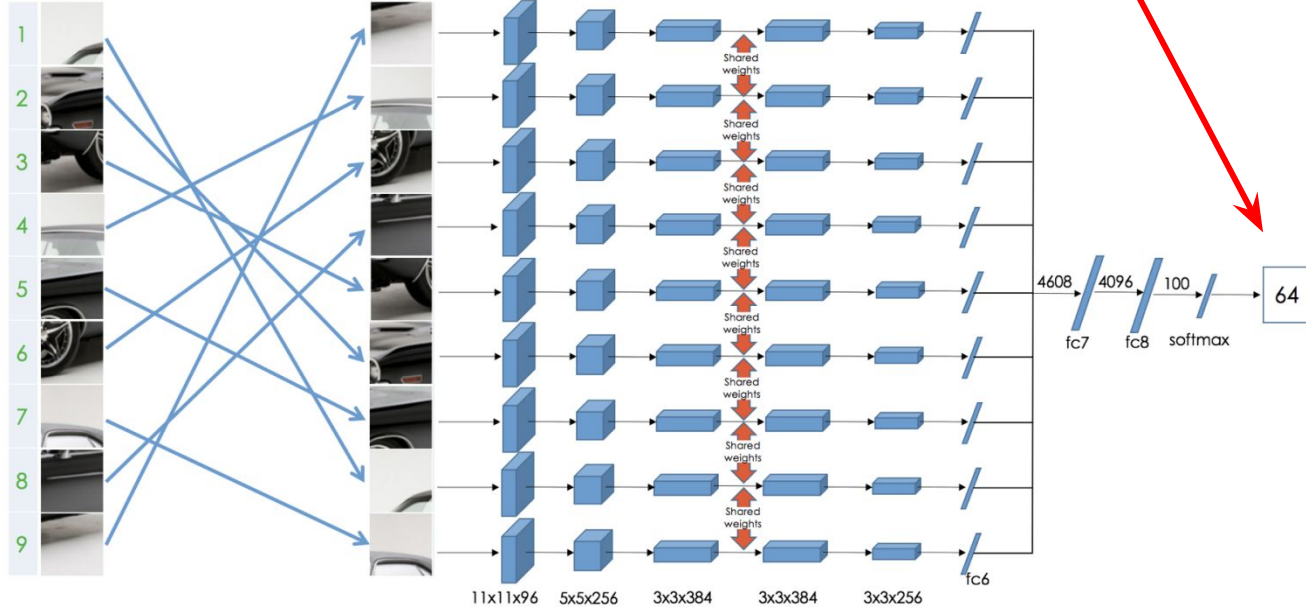
Fixed set of permutations
Cross-entropy loss



Permutation Set

index	permutation
64	9,4,6,8,3,2,5,1,7

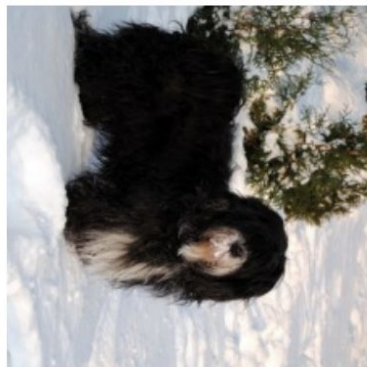
Reorder patches according to the selected permutation



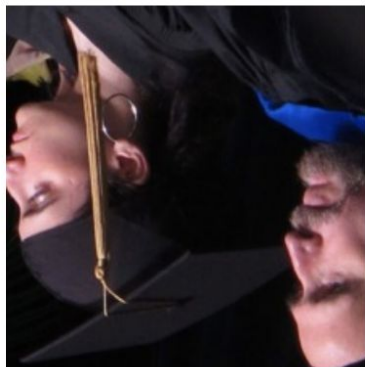
Rotation (Gidaris 2018)



90° rotation



270° rotation



180° rotation

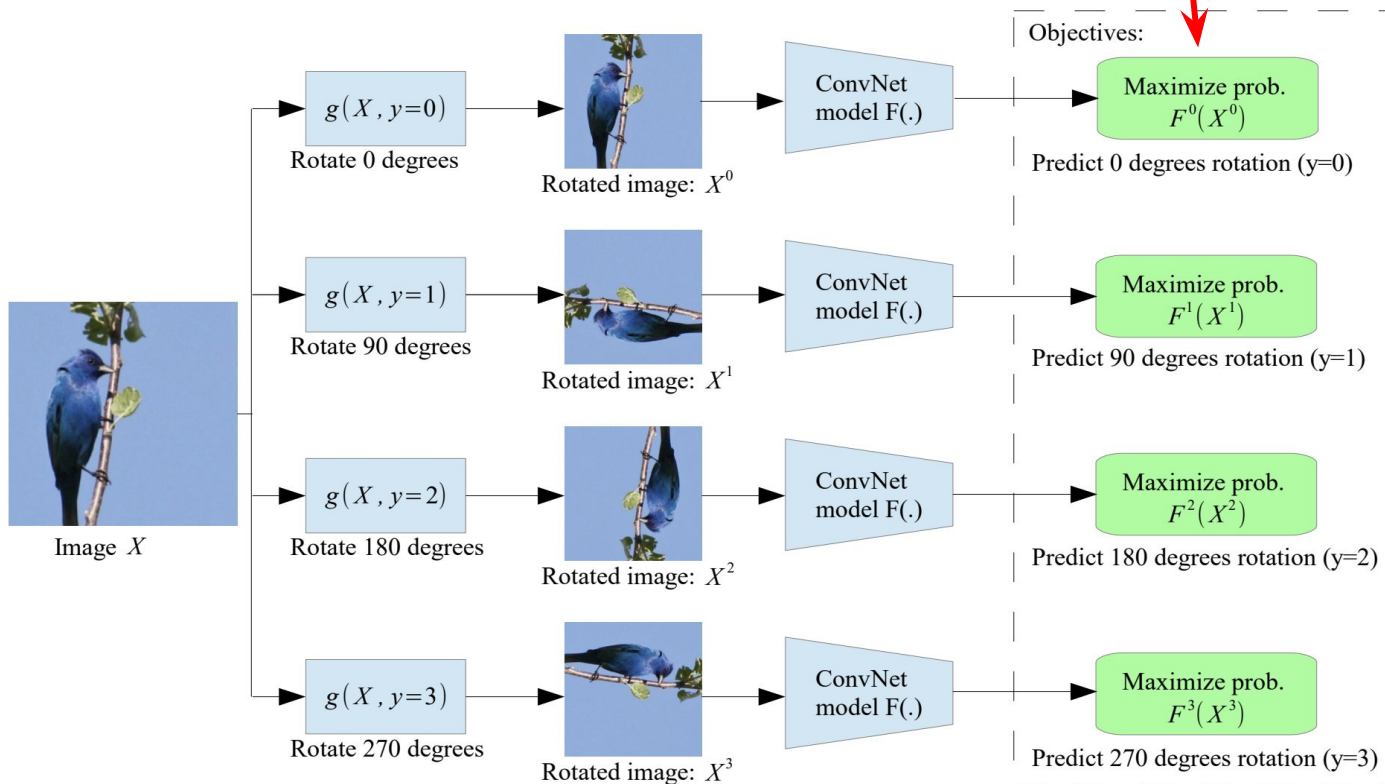


0° rotation



270° rotation

Rotation (Gidaris 2018)



Rotation (Gidaris 2018)

# Rotations	Rotations	CIFAR-10 Classification Accuracy
4	0°, 90°, 180°, 270°	89.06
8	0°, 45°, 90°, 135°, 180°, 225°, 270°, 315°	88.51
2	0°, 180°	87.46
2	90°, 270°	85.52

Train small MLP on top of fixed feature extractor

Rotation (Gidaris 2018)

Method	Conv4	Conv5
ImageNet labels from (Bojanowski & Joulin, 2017)	59.7	59.7
Random from (Noroozi & Favaro, 2016)	27.1	12.0
Tracking Wang & Gupta (2015)	38.8	29.8
Context (Doersch et al., 2015)	45.6	30.4
Colorization (Zhang et al., 2016a)	40.7	35.2
Jigsaw Puzzles (Noroozi & Favaro, 2016)	45.3	34.6
BIGAN (Donahue et al., 2016)	41.9	32.2
NAT (Bojanowski & Joulin, 2017)	-	36.0
(Ours) RotNet	50.0	43.8

Train small non-linear classifier on top of indicated layer of fixed feature extractor

Common-Sense Visual Tasks

Limitations:

Tasks are very one-dimensional

Models are trained on weird inputs

Approaches to SSL

- Reconstructing corrupted images
- Autoregressive
- Common-sense visual tasks
- **Contrastive learning**
- Feature prediction and self-distillation
- Text-image (CLIP)

Contrastive Learning

Push features of similar images together

Pull features of distinct images apart

Positive pairs - similar images

Negative pairs - distinct images

Need:

(1) way to generate or define positive/negative pairs

(2) loss that squeezes features for positive pairs and pulls negative pairs apart

Contrastive Learning

Supervised classification:

Positive pairs - images from same class

Negative pairs - images from different classes

Self-supervised pretraining:

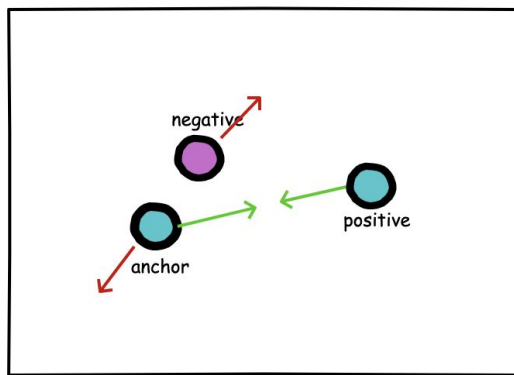
Positive pairs - augmented views of the same base image

Negative pairs - views of different base images

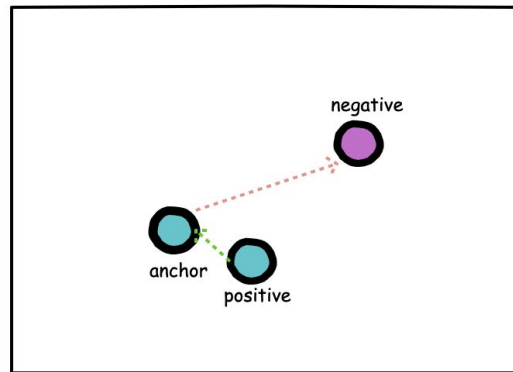
Example Loss Function - Triplet Loss

$$\mathcal{L}(A, P, N) = \max(\|f(A) - f(P)\|_2 - \|f(A) - f(N)\|_2 + \alpha, 0)$$

where A is an *anchor input*, P is a *positive input* of the same class as A , N is a *negative input* of a different class from A , α is a margin between positive and negative pairs, and f is an embedding.



Embeddings before triplet loss update



Embeddings after triplet loss update



MoCo (He 2020)

Two augmented views per sample in current batch, extract features. Keep queue of features extracted from previous batches

Positive pair: Features from two augmented views of same image in current batch

Negative pair: Features from one image in the current batch and one in the queue

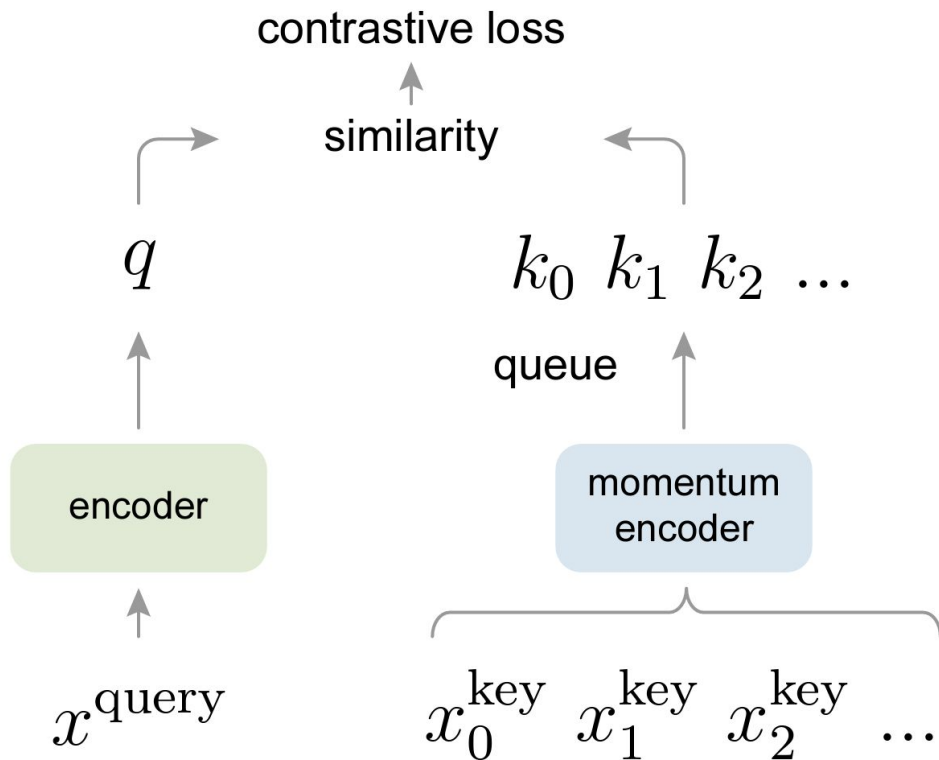
In every pair, extract features from one image using current feature extractor, from other using momentum moving average.

Queue size independent of batch size

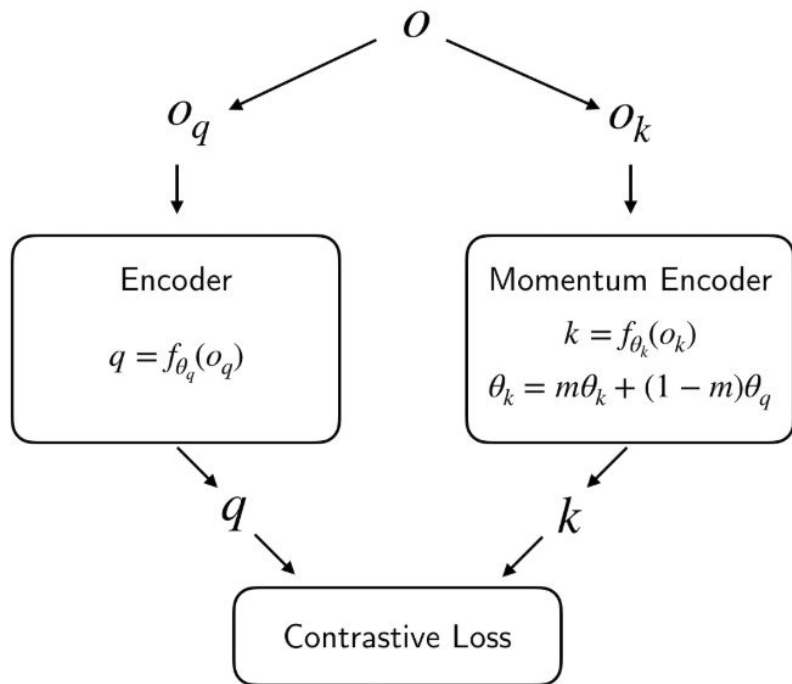
Augmentations:

- Random resize crop
- Color jitter - Randomly change the brightness, contrast, saturation, and hue
- Horizontal flip
- Random grayscale conversion

MoCo (He 2020)



MoCo (He 2020)



$$\mathcal{L}_q = -\log \frac{\exp(q \cdot k_+ / \tau)}{\sum_{i=0}^K \exp(q \cdot k_i / \tau)}$$

View as softmax crossentropy.
Temperature tunes concentration
of probability distribution

```

# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
    x_q = aug(x) # a randomly augmented version
    x_k = aug(x) # another randomly augmented version

    q = f_q.forward(x_q) # queries: NxK
    k = f_k.forward(x_k) # keys: NxK
    k = k.detach() # no gradient to keys

    # positive logits: Nx1
    l_pos = bmm(q.view(N,1,K), k.view(N,K,1))

    # negative logits: NxK
    l_neg = mm(q.view(N,K), queue.view(K,K))

    # logits: Nx(1+K)
    logits = cat([l_pos, l_neg], dim=1)

    # contrastive loss, Eqn. (1)
    labels = zeros(N) # positives are the 0-th
    loss = CrossEntropyLoss(logits/t, labels)

    # SGD update: query network
    loss.backward()
    update(f_q.params)

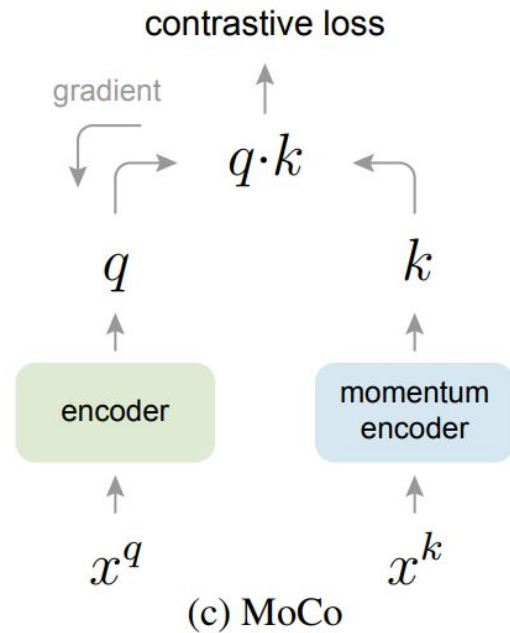
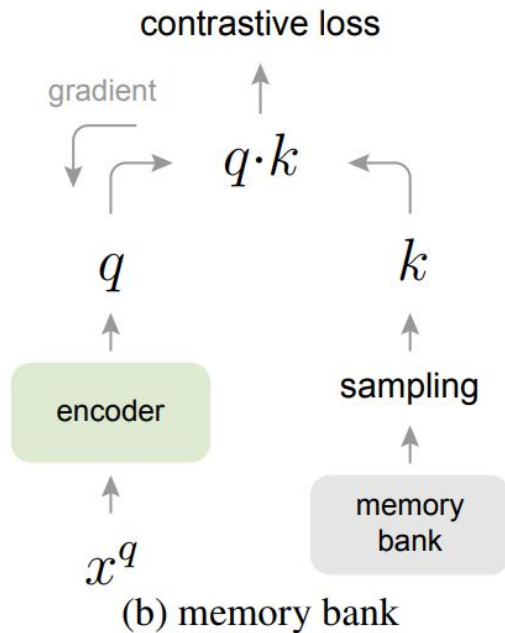
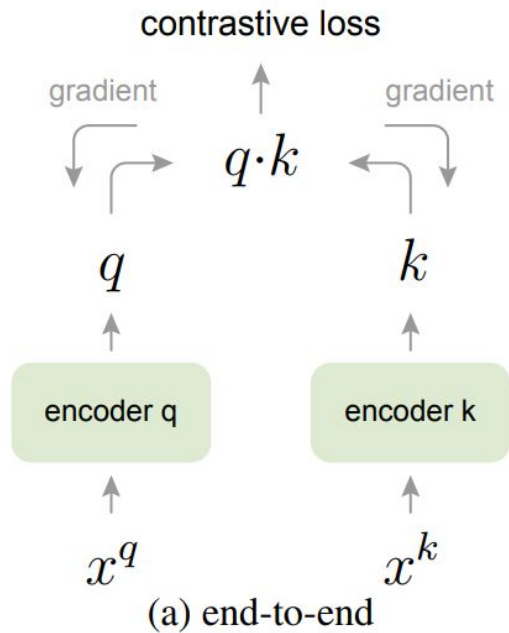
    # momentum update: key network
    f_k.params = m*f_k.params+(1-m)*f_q.params

    # update dictionary
    enqueue(queue, k) # enqueue the current minibatch
    dequeue(queue) # dequeue the earliest minibatch

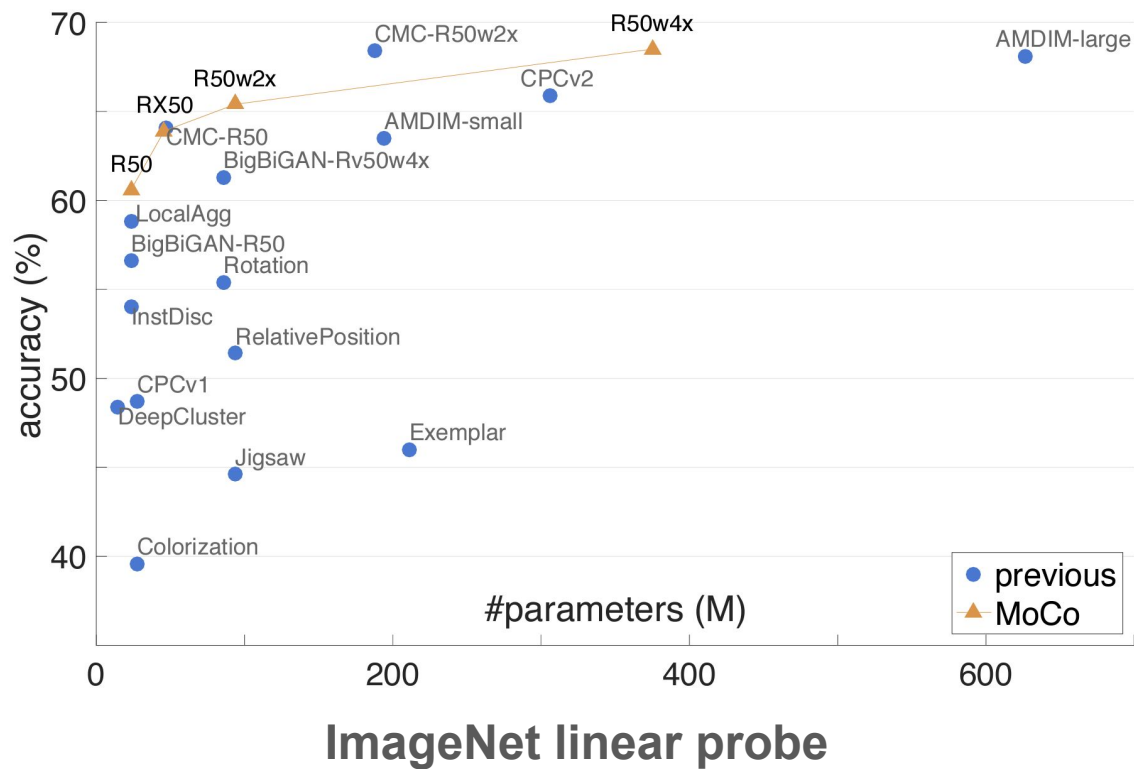
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

MoCo (He 2020)



MoCo (He 2020)



SimCLR (Chen 2020)

Much simpler

✗ Get rid of momentum moving average

✗ Get rid of queue

✓ Same feature extractor for positive/negative examples

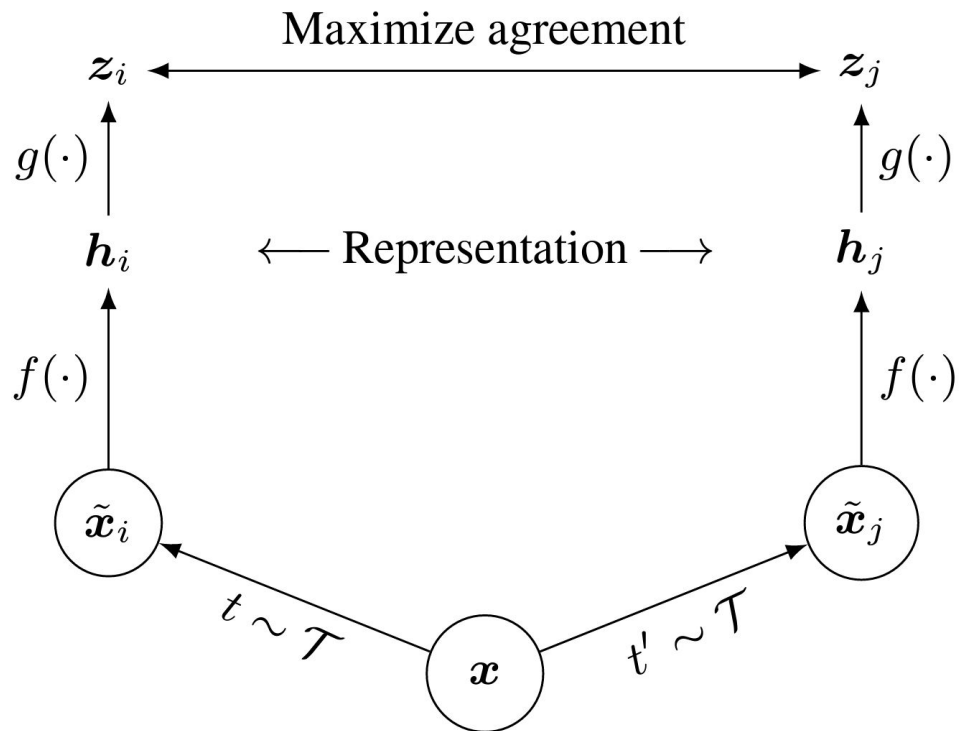
✓ Negative samples come from same batch

Add on small MLP projection head on top of convnet, tear off after pretraining

Huge batch sizes

Incorporate Gaussian blur augmentation, different augmentation rates

SimCLR (Chen 2020)



SimCLR (Chen 2020)

Loss function for positive pair:

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

Cosine similarity instead of dot product

Temperature tau

Normalization + temp restricts range of similarity scores

SimCLR (Chen 2020)

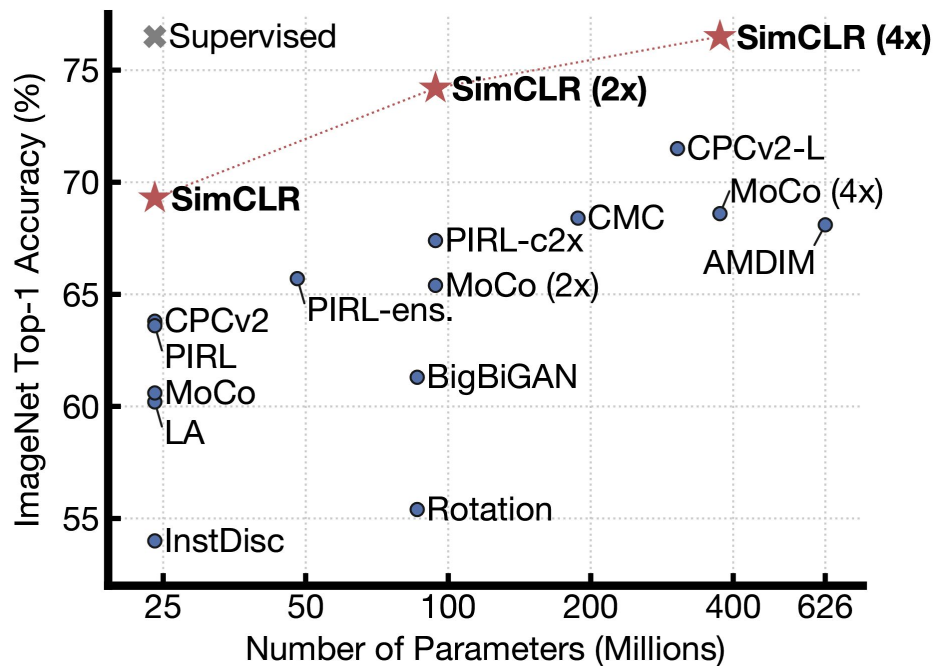
ℓ_2 norm?	τ	Entropy	Contrastive acc.	Top 1
Yes	0.05	1.0	90.5	59.7
	0.1	4.5	87.8	64.4
	0.5	8.2	68.2	60.7
	1	8.3	59.1	58.0
No	10	0.5	91.7	57.2
	100	0.5	92.1	57.0

Table 5. Linear evaluation for models trained with different choices of ℓ_2 norm and temperature τ for NT-Xent loss. The contrastive distribution is over 4096 examples.

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N , constant τ , structure of $f, g, \mathcal{T}$.
for sampled minibatch $\{\mathbf{x}_k\}_{k=1}^N$ **do**
 for all $k \in \{1, \dots, N\}$ **do**
 draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
 # the first augmentation
 $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$
 $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$ # representation
 $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$ # projection
 # the second augmentation
 $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$
 $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$ # representation
 $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$ # projection
 end for
 for all $i \in \{1, \dots, 2N\}$ and $j \in \{1, \dots, 2N\}$ **do**
 $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$ # pairwise similarity
 end for
 define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
 $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
 update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$

SimCLR (Chen 2020)



Linear probe, ResNet-50

MoCo v2/v3

v2: MoCo (v1) with modifications from SimCLR

- Add projection head
- Similar augmentation policy to SimCLR, add Gaussian blur
- Cosine learning rate schedule

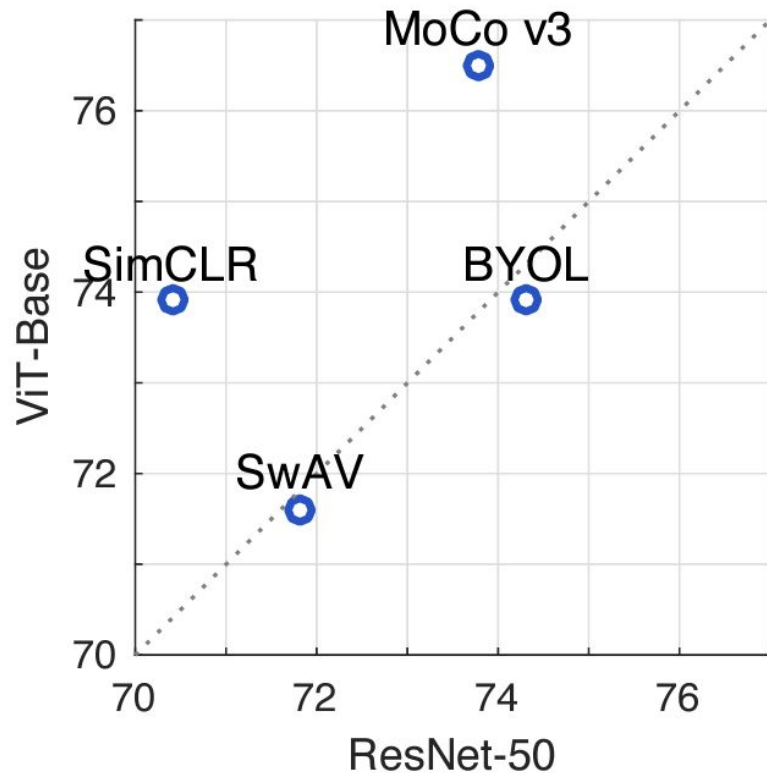
v3: MoCo v2 with modifications:

- Apply v2 to vision transformers
- Get rid of queue, just use negative samples within the batch
- Encoder (non-momentum) uses extra prediction head, symmetry breaker
- Big batch sizes + lots of hyperparameter tuning to stabilizing training

MoCo v2/v3

case	unsup. pre-train					ImageNet acc.
	MLP	aug+	cos	epochs	batch	
MoCo v1 [6]				200	256	60.6
SimCLR [2]	✓	✓	✓	200	256	61.9
SimCLR [2]	✓	✓	✓	200	8192	66.6
MoCo v2	✓	✓	✓	200	256	67.5
<i>results of longer unsupervised training follow:</i>						
SimCLR [2]	✓	✓	✓	1000	4096	69.3
MoCo v2	✓	✓	✓	800	256	71.1

ImageNet linear probe



Contrastive Learning vs. Meta-Learning (Ni 2022)

Contrastive learning $\approx$ meta-learning with the right few-shot classification task

Sample two augmentations of each of n images

Use one of each as query one of each as support

n -way 1-shot classification, each class is augmentations of one image

Contrastive loss $\approx$ ProtoNets

Use meta-learning tricks to boost contrastive learning performance and vice versa

Contrastive Learning vs. Meta-Learning (Ni 2022)

Table 3: ImageNet Top-1 accuracy (%) of models fine-tuned with few labels.

Method	Backbone	Label fraction	
		1%	10%
Supervised baseline	ResNet-50	25.4	56.4
SimCLR	ResNet-50	32.4	53.6
ProtoNet	ResNet-50	31.0	52.9
R2-D2	ResNet-50	37.9	58.8

Contrastive Learning vs. Meta-Learning (Ni 2022)

Algorithm 1: Meta-Learning Framework for Self-Supervised Learning

Require: Base model F_θ , classification strategy $\mathcal{A}$, learning rate γ , and distribution of data augmentations $\mathcal{D}$.

Initialize θ , the weights of F ;

while *not done* **do**

for $j = 1, \dots, n$ **do**

 Sample a batch of base images $\mathbf{x}$;

 Sample m random data augmentations from $\mathcal{D}$ to obtain augmented views $\{\tilde{\mathbf{x}}^i\}_{i=1}^m$;

 Separate $\{\tilde{\mathbf{x}}^i\}_{i=1}^m$ into support set $\mathcal{T}_j^s = \{\tilde{\mathbf{x}}^i\}_{i=1}^{m_1}$ and query set $\mathcal{T}_j^q = \{\tilde{\mathbf{x}}^i\}_{i=m_1+1}^m$;

 Fine-tune model on $\mathcal{T}_j$: $\theta_j = \mathcal{A}(\theta, \mathcal{T}_j^s)$;

 Compute gradient $g_j = \nabla_\theta \mathcal{L}(F_{\theta_j}, \mathcal{T}_j^q)$;

end

 Update base model parameters: $\theta \leftarrow \theta - \frac{\gamma}{n} \sum_j g_j$.

end

Approaches to SSL

- Reconstructing corrupted images
- Autoregressive
- Common-sense visual tasks
- Contrastive learning
- Feature prediction and self-distillation
- Text-image (CLIP)

Feature Prediction and Self-Distillation

Contrastive learning - pull together similar samples, push apart distinct ones

Imagine objective that only maximizes feature similarity of similar samples
(no negative pairs)

Degenerate solution: make every input map to the same feature vector, then everything is similar to everything else.

How to train without negative pairs while avoiding collapse?

Why? Could train with smaller batch sizes while avoiding memory queue.

BYOL (Grill 2020)

Similar to MoCo v3

- Two augmented views of each image
- Momentum exponential moving average
- Feature extractor + projectors + prediction head
- Normalize features (as in cosine similarity)

Loss function: Just make features for the two augmented views of the same image similar

BYOL (Grill 2020)

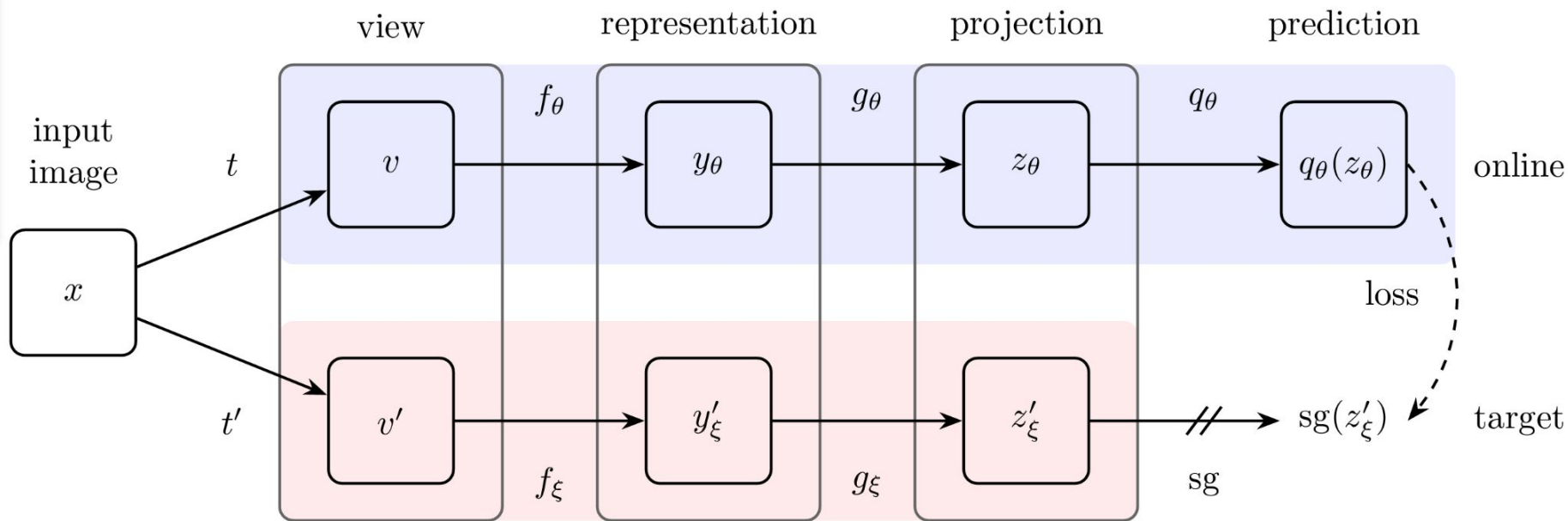
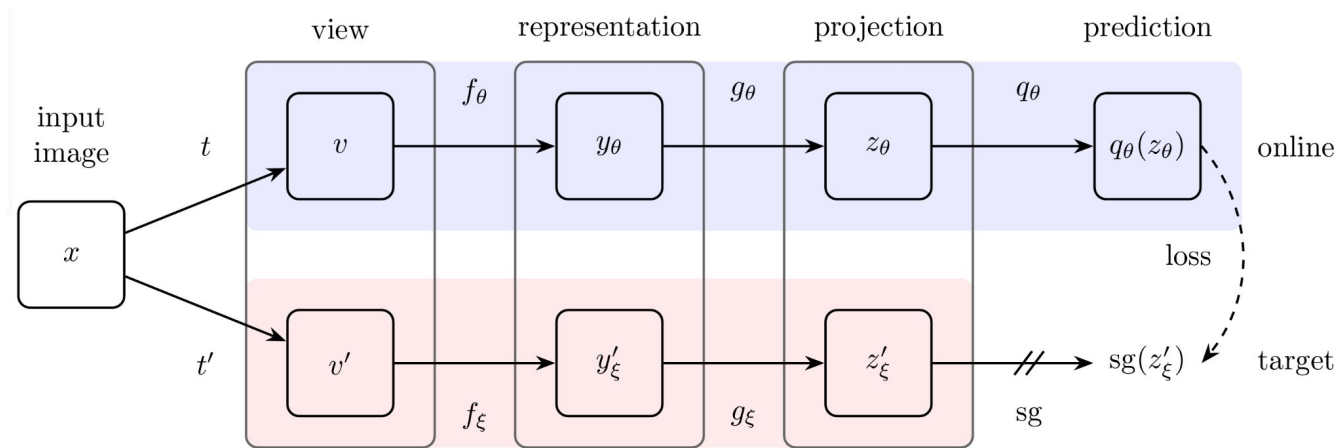


Figure 2: BYOL's architecture. BYOL minimizes a similarity loss between $q_\theta(z_\theta)$ and $sg(z'_\xi)$, where θ are the trained weights, ξ are an exponential moving average of θ and sg means stop-gradient. At the end of training, everything but f_θ is discarded, and y_θ is used as the image representation.

BYOL (Grill 2020)



$$\bar{z}'_\xi \triangleq z'_\xi / \|z'_\xi\|_2$$

$$\overline{q_\theta}(z_\theta) \triangleq q_\theta(z_\theta) / \|q_\theta(z_\theta)\|_2$$

$$\mathcal{L}_{\theta,\xi} \triangleq \|\overline{q_\theta}(z_\theta) - \bar{z}'_\xi\|_2^2 = 2 - 2 \cdot \frac{\langle q_\theta(z_\theta), z'_\xi \rangle}{\|q_\theta(z_\theta)\|_2 \cdot \|z'_\xi\|_2}$$

BYOL (Grill 2020)

Stop gradient prevents generate solution (feature collapse)

Moving target - changing the parameters to move the features from image A towards features from positive pair-mate A' will move the features for A'

Understanding the effects of stop gradient is a research topic

BYOL (Grill 2020)

Method	Top-1	Top-5
Local Agg.	60.2	-
PIRL [35]	63.6	-
CPC v2 [32]	63.8	85.3
CMC [11]	66.2	87.0
SimCLR [8]	69.3	89.0
MoCo v2 [37]	71.1	-
InfoMin Aug. [12]	73.0	91.1
BYOL (ours)	74.3	91.6

ImageNet linear probe, ResNet-50

BYOL (Grill 2020)

Don't need big batch sizes

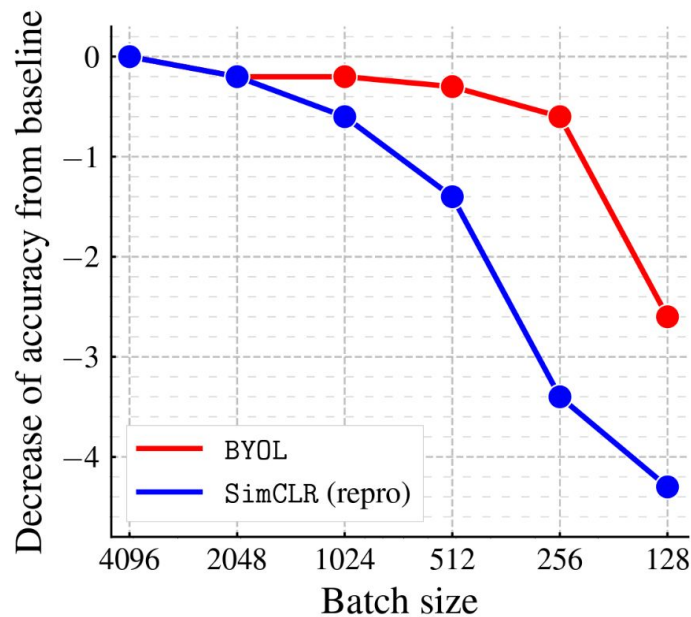
Loss - MSE between positive pairs, sum over all pairs in batch

Sum of loss for two small batches is same as loss for one big batch

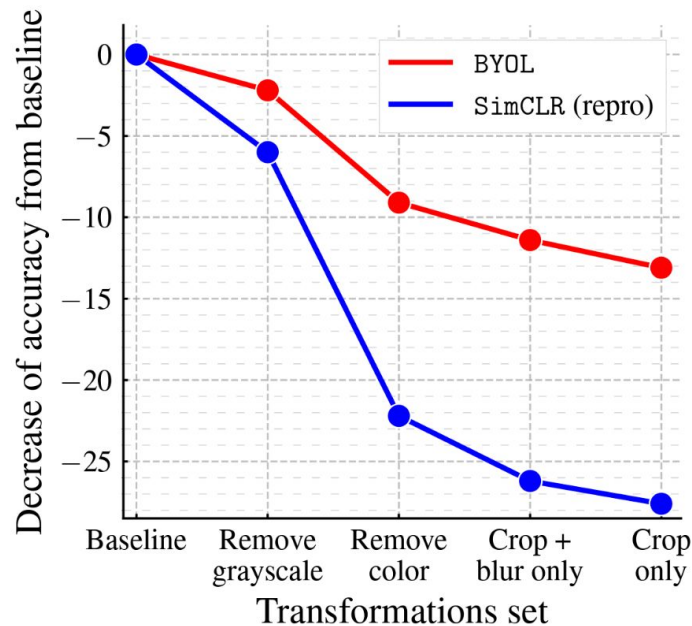
Not true for contrastive learning

Contrastive learning needs lots of negative samples so SimCLR used huge batches

BYOL (Grill 2020)



(a) Impact of batch size



(b) Impact of progressively removing transformations

Figure 3: Decrease in top-1 accuracy (in % points) of BYOL and our own reproduction of SimCLR at 300 epochs, under linear evaluation on ImageNet.

SimSiam (Chen 2020)

What do we need to avoid feature collapse and get good performance?

✗ No momentum moving average

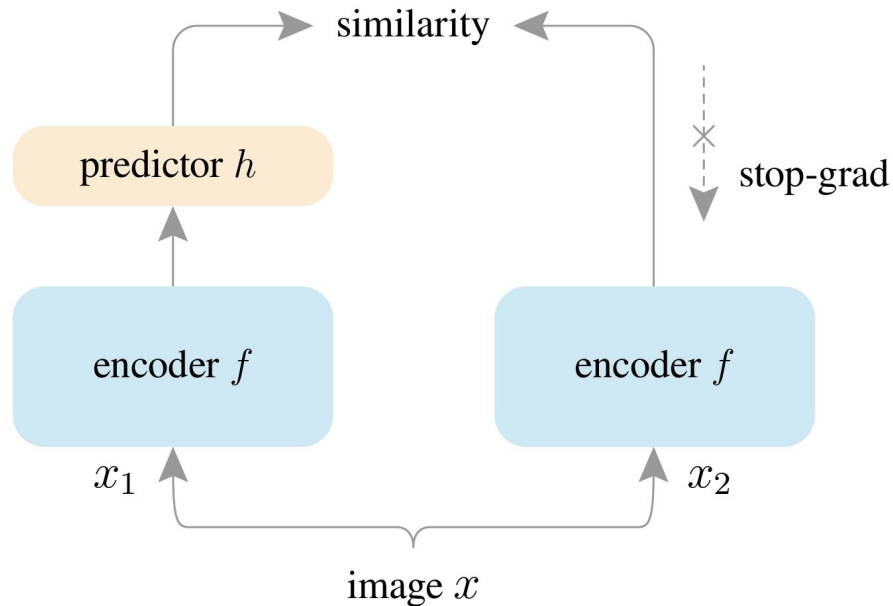
✗ No negative samples

✗ No big batches

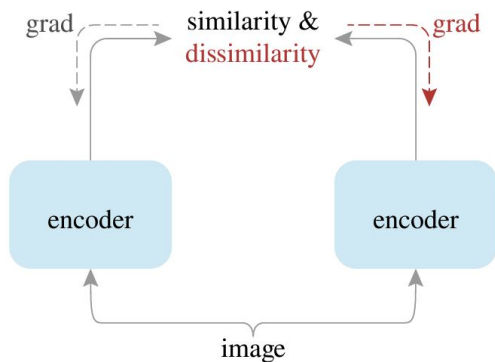
✓ stop gradient

✓ encoder + projector + prediction head

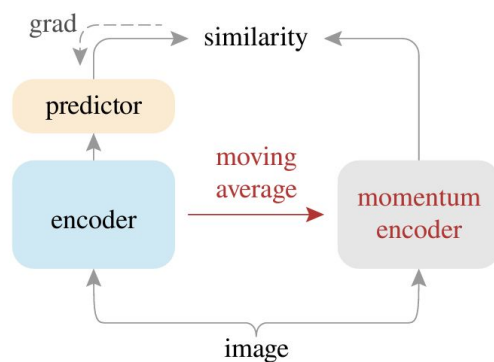
✓ cosine similarity loss



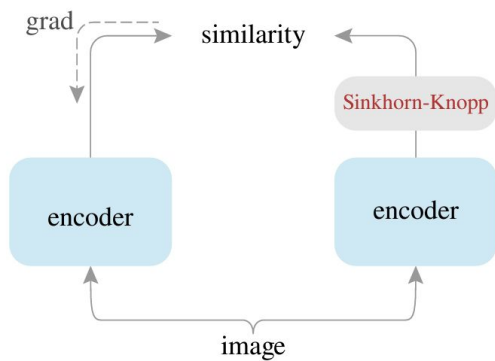
SimSiam (Chen 2020)



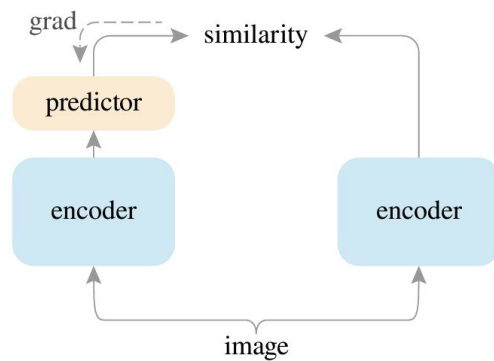
SimCLR



BYOL



SwAV



SimSiam

Advantages and Disadvantages of SSL Approaches

Contrastive learning, feature prediction and self-distillation perform SOTA

But ... they require high-quality data augmentations

MAE doesn't perform quite as well usually, especially without fine-tuning

But ... masking generalizes much better to other data modalities (e.g. language, audio, etc.)

A Quick Point about SSL for Vision

Promise of SSL is training on large uncurated datasets where labeling is expensive

In practice, most papers train on ImageNet where we have labels and images are class-balanced with ~ 1 object per image

Doesn't tell us if SSL works in the wild

SEER (Goyal 2021)

Train on 1 billion “random images

Use SwAV, kind of like SimCLR but uses clustering instead of tons of pairwise comparisons

SSL benefits from more data, even in the wild

Not as big gains as in NLP

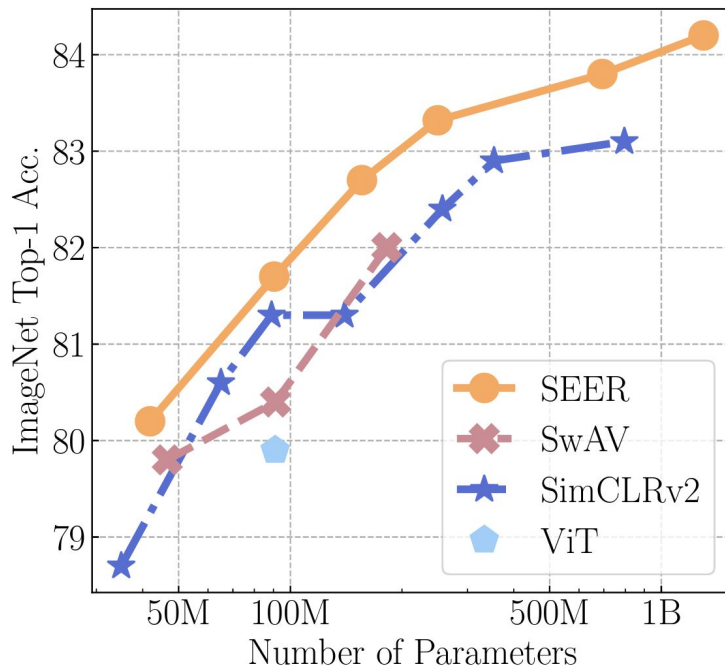


Figure 1: Performance of large pretrained models on ImageNet. We pretrain our SEER models on an uncurated and random images. They are RegNet architectures [40] trained with the SwAV self-supervised method [7]. We compare with the original models trained in Caron et al. [7] as well as the pretraining on curated data from SimCLRv2 [9] and ViT [14]. The network architectures are different. We report the top-1 accuracy after finetuning on ImageNet.

Approaches to SSL

- Reconstructing corrupted images
- Autoregressive
- Common-sense visual tasks
- Contrastive learning
- Feature prediction and self-distillation
- Text-image (CLIP)

CLIP (Radford 2021)

We previously saw LLMs

And now we saw SSL for vision

Can we train models that understand both images and text?

CLIP:

- Map images and text to a common feature space
- Images and text that describe them have similar features
- Use contrastive learning
- Massive captioned image dataset
- Positive pairs - image + caption
- Negative pairs - image + different caption

CLIP (Radford 2021)

Dataset:

Generate 500k search queries - mostly English Wikipedia titles, commonly occurring words and phrases from Wikipedia

400m image/caption pairs, up to 20k per query

Similar total number of words to GPT-2 pretraining dataset, WebText

Noisy captions, but washes out over massive training set

Closed source

CLIP (Radford 2021)

Backend url:

Index:

[Clip retrieval](#) works by converting the text query to a CLIP embedding, then using that embedding to query a knn index of clip image embeddings

Display captions ☒
Display full captions ☐
Display similarities ☐
Safe mode ☒
Hide duplicate urls ☒
Search over

This UI may contain results with nudity and is best used by adults. The images are under their own copyright.

Are you seeing near duplicates ? KNN search are good at spotting those, especially so in large datasets.



Fluffy Kitten does not know what to do.



Best Cute Kitten Wallpaper No 5



cutestofthecute: (via)



5D Diamond Painting White Cat with Blue Eyes Kit



Cute White Cat Hd



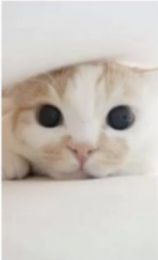
...cute little kittie... :)



Criadero especializado en British Shorthair



Gorgeous Himalayan Persian Kittens



Cats are one of the few



Fluffy Orange Kitten With Blue Eyes | Too Cute!



Cute cat wallpaper



This Munchkin Kitten Will Melt Your Heart With Cut...



This Cat Has the Most Beautiful Eyes - We Love Cat...



Snoopy, Exotic Shorthair.

LAION - open source alternative datasets, used to train OpenCLIP

CLIP (Radford 2021)

**Don't need data
augmentation when you
train on so much data**

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t            - learned temperature parameter
```

**Linear projectors
Don't use non-linear ones**

**Temperature is learned (treated as
param) but clipped to keep logits
from getting scaled too high**

```
# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]
```

```
# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)
```

Normalize

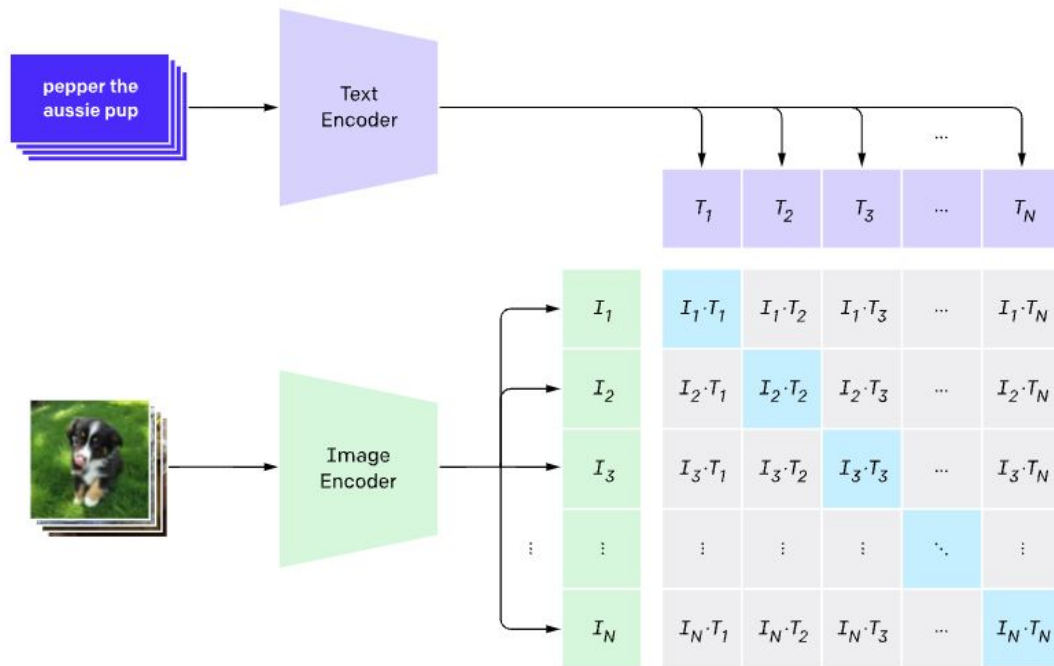
```
# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)
```

**Maximize cosine similarity
of positive pairs (N Pairs)
Minimize cosine similarity of
negative pairs ($N^2 - N$ pairs)**

```
# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

CLIP (Radford 2021)

1. Contrastive pre-training



CLIP (Radford 2021)

How to use CLIP after pretraining?

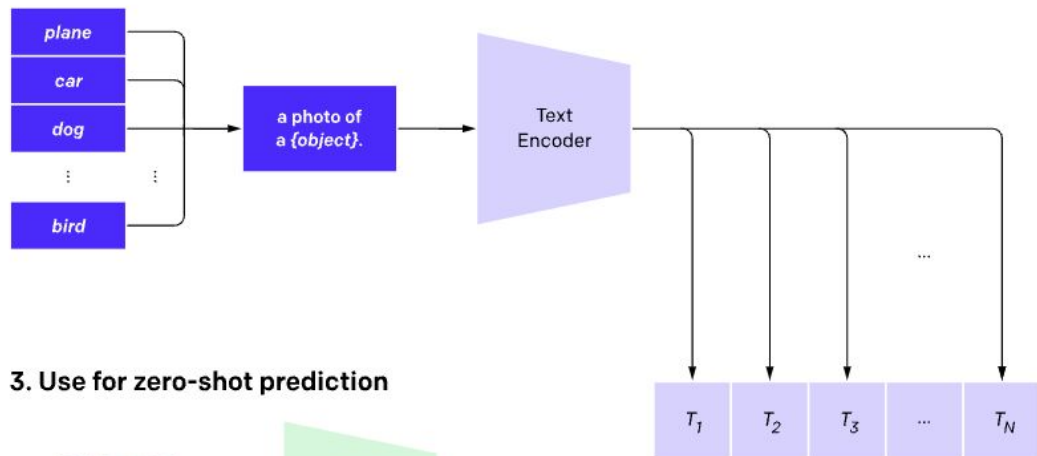
Can just use image feature extractor - e.g. fine-tuning

Can just use text feature extractor - e.g. plug features into diffusion models

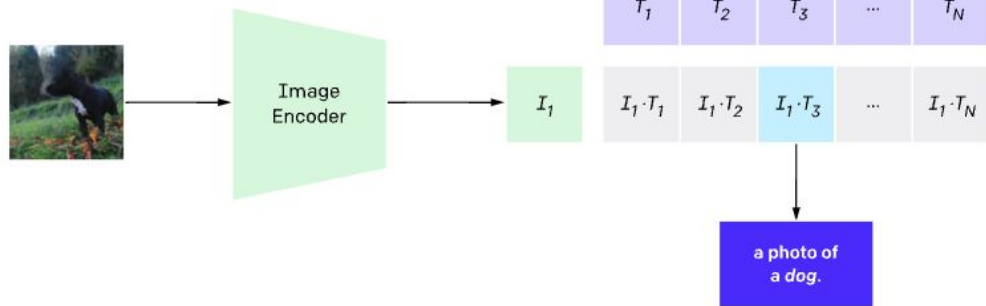
Zero-shot image classification

CLIP (Radford 2021)

2. Create dataset classifier from label text



3. Use for zero-shot prediction



CLIP (Radford 2021)

Food101

guacamole (90.1%) Ranked 1 out of 101 labels



- ✓ a photo of **guacamole**, a type of food.
- ✗ a photo of ceviche, a type of food.
- ✗ a photo of edamame, a type of food.
- ✗ a photo of tuna tartare, a type of food.
- ✗ a photo of hummus, a type of food.

Youtube-BB

airplane, person (89.0%) Ranked 1 out of 23 labels



- ✓ a photo of a **airplane**.
- ✗ a photo of a bird.
- ✗ a photo of a bear.
- ✗ a photo of a giraffe.
- ✗ a photo of a car.

SUN397

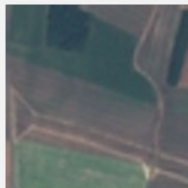
television studio (90.2%) Ranked 1 out of 397 labels



- ✓ a photo of a **television studio**.
- ✗ a photo of a podium indoor.
- ✗ a photo of a conference room.
- ✗ a photo of a lecture room.
- ✗ a photo of a control room.

EuroSAT

annual crop land (46.5%) Ranked 4 out of 10 labels



- ✗ a centered satellite photo of permanent crop land.
- ✗ a centered satellite photo of pasture land.
- ✗ a centered satellite photo of highway or road.
- ✓ a centered satellite photo of **annual crop land**.
- ✗ a centered satellite photo of brushland or shrubland.

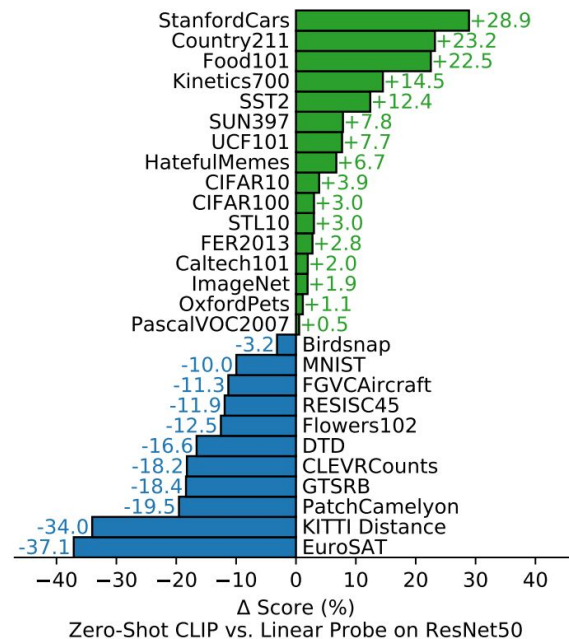
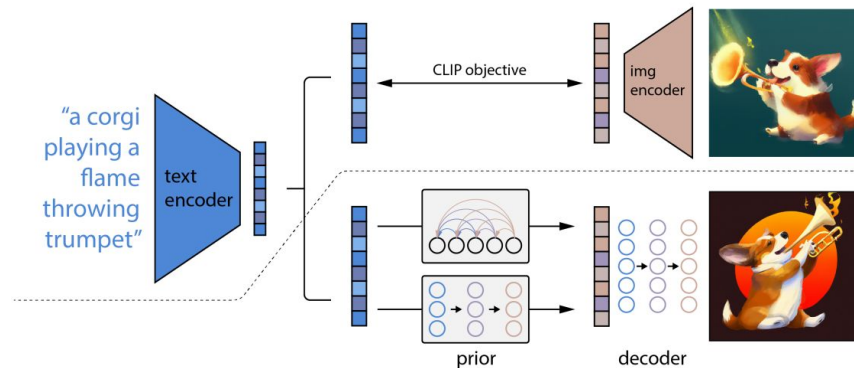


Figure 5. Zero-shot CLIP is competitive with a fully supervised baseline. Across a 27 dataset eval suite, a zero-shot CLIP classifier outperforms a fully supervised linear classifier fitted on ResNet-50 features on 16 datasets, including ImageNet.

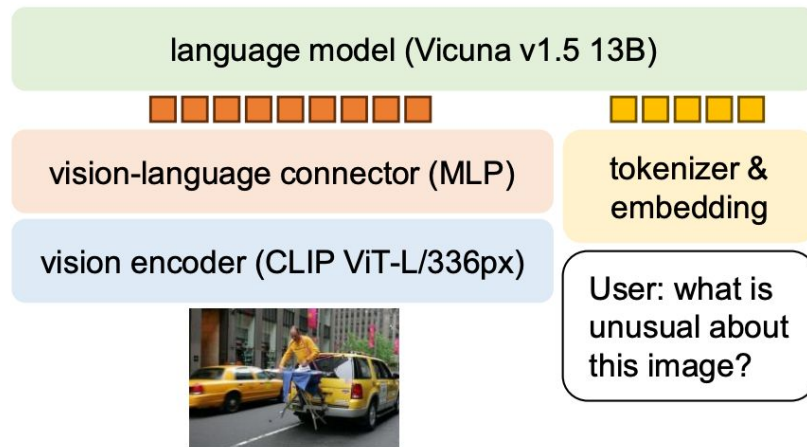
CLIP (Radford 2021)

CLIP learns useful features for other models



unCLIP

CLIP text features -> diffusion

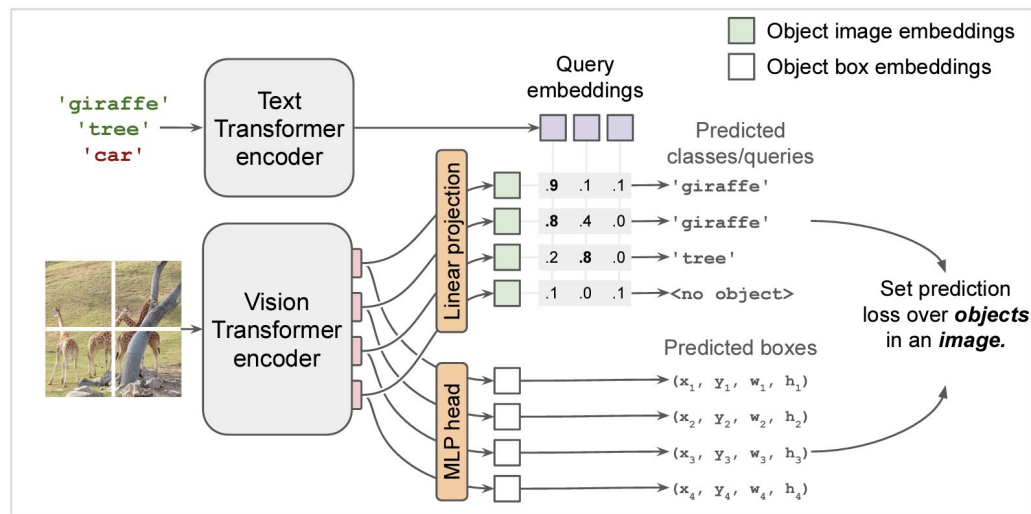


LLaVa

CLIP vision encoder, plug images into LLMs

CLIP (Radford 2021)

CLIP learns useful features for other models



OWL-ViT

Open vocabulary object detection

What kinds of features does CLIP learn? (Ghiasi 2022)

Training on images alone -> similar features for images with similar appearance

CLIP assigns similar features to images that contain semantically similar content.



(a) Category of morbidity

(b) Category of music

Figure 11: **Features from ViT trained with CLIP that relates to the category of morbidity and music.** *Top-left image in each category: Image optimized to maximally activate a feature from layer 10. Rest: Seven of the ten ImageNet images that most activate the feature.*

Evals - Battle of the Backbones

Tasks:

- Classification
- Detection and segmentation
- Retrieval
- OOD (e.g. sim2real)

Pretraining strategies:

- Contrastive SSL
- Feature prediction SSL
- CLIP
- Generative models - Stable Diffusion
- Supervised - classification, monocular depth estimation
- From scratch

Architectures:

- CNNs (old to modern)
- ViTs (old to modern)

Evals - Battle of the Backbones

Relevant takeaways:

- SSL pretraining > supervised pretraining on downstream tasks for same amount of pretraining data, but can find supervised models with more data
- Transformers (e.g. MAE) benefit much more from fine-tuning, bad off the shelf
- CLIP great across downstream tasks
- SSL and supervised >> generative backbones
- SSL models are more adversarially robust

Evals - Battle of the Backbones

Performance is correlated across tasks

